FIRESENTRY : MITIGASI KEBAKARAN DINI BERBASIS IOT

EMBEDDED SYSTEM



DISUSUN OLEH :

REZKY AULIASARIE	- 245150300111014
AGATHA TRIOTAMA	- 245150300111020
ZEYRA RAHMA ALIVIA	- 245150301111016
MUHAMMAD NAUFAL M. R	- 245150307111006
DELVIN NHEAN OLAMINA	- 245150307111032

FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA

2026

DAFTAR ISI

BAB I **2**
PENDAHULUAN **2**
1.1 LATAR BELAKANG 2
1.2 RUMUSAN MASALAH 3
1.3 TUJUAN 3
1.4 MANFAAT 3
BAB II **5**
PERANCANGAN SISTEM **5**
2.1 DESIGN FLOW 5
2.2 STATECHART DAN UML 6
2.2.1 USE CASE DIAGRAM 6
2.2.2 SEQUENCE DIAGRAM 8
2.2.3 STATECHART DIAGRAM 10
2.3 INTERFACING DENGAN LINGKUNGAN 12
2.3.1 SENSOR 12
2.3.2 INPUT DIGITAL DAN ANALOG 13
2.3.3 SIGNAL 13
2.3.4 SAMPLING DAN ALIASING 14
2.3.5 ADC (Analog to Digital Converter) dan Resolusi 14
2.3.6 FILTERING 15
2.4 PROCESSING HARDWARE 15
2.4.1 ALAT DAN BAHAN 15
2.4.2 DESAIN SISTEM 18
2.4.3 SKEMATIK HARDWARE 19
2.5 KOMUNIKASI DATA 21
2.5.1 KOMUNIKASI IOT (CLOUD & TELEGRAM BOT) 21
2.5.2 PROTOKOL KOMUNIKASI HARDWARE 22
2.6 MESSAGE QUEUING TELEMETRY TRANSPORT (MQTT) 23
BAB III **25**
IMPLEMENTASI SISTEM **25**
3.1 IMPLEMENTASI HARDWARE 25
3.2 IMPLEMENTASI SOFTWARE 25
3.3 INTEGRASI SENSOR 38
3.4 IMPLEMENTASI MONITORING DAN NOTIFIKASI TELEGRAM BOT 40
BAB IV **42**
ANALISIS DAN PENGUJIAN **42**
4.1 RELIABILITY 42
4.2 VALIDASI DAN EVALUASI 43
4.3 OPTIMASI SISTEM 44
4.4 TESTING 46
4.4.1 TABEL PENGUJIAN JARAK DETEKSI FLAME IR SENSOR 46
4.4.2 TABEL PENGUJIAN SENSOR MQ-2 BERDASARKAN JARAK ASAP/GAS 46
4.4.3 TABEL PENGUJIAN SENSOR SUHU DS18B20 BERDASARKAN PERUBAHAN TEMPERATUR 47
4.4 TABEL PENGUJIAN KOMBINASI SENSOR 48
4.4.5 TABEL PENGUJIAN NOTIFIKASI IOT (MQTT/TELEGRAM) 48
4.4.6 TABEL PENGUJIAN RELIABILITY SISTEM 49
4.4.7 PERBANDINGAN MQTT VS TELEGRAM BOT API 50
BAB V **50**
PENUTUP **51**
5.1 KESIMPULAN 51
5.2 SARAN 51

BAB I

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi informasi saat ini mengarah pada konsep ubiquitous computing dan cyber-physical systems, di mana sistem komputasi tidak lagi berdiri sendiri, melainkan tertanam di dalam perangkat dan lingkungan untuk menjalankan fungsi tertentu secara otomatis. Embedded system menjadi salah satu teknologi utama yang mendukung perkembangan tersebut karena mampu mengintegrasikan perangkat keras, perangkat lunak, serta lingkungan fisik dalam suatu sistem yang bekerja secara spesifik dan real-time. Berdasarkan definisinya, embedded system merupakan sistem komputer khusus yang dirancang untuk menjalankan satu atau beberapa fungsi tertentu dengan batasan real-time serta melibatkan komponen hardware dan software secara terintegrasi.

Penerapan embedded system telah digunakan di berbagai bidang, seperti automotive electronics, medical systems, smart home appliances, hingga sistem kontrol industri. Hal ini menunjukkan bahwa embedded system memiliki peran penting dalam meningkatkan efisiensi, keamanan, dan otomatisasi pada kehidupan sehari-hari. Salah satu permasalahan yang masih sering terjadi di lingkungan masyarakat adalah kebakaran rumah atau bangunan akibat korsleting listrik, kebocoran gas, maupun keterlambatan penanganan sumber api. Pada umumnya, sistem pendeteksi kebakaran konvensional hanya memberikan alarm peringatan tanpa mampu melakukan pemantauan kondisi lingkungan secara menyeluruh dan penyampaian informasi secara real-time kepada pengguna.

Berdasarkan permasalahan tersebut, diperlukan suatu sistem embedded yang mampu melakukan pemantauan lingkungan secara real-time, mendeteksi potensi kebakaran secara akurat, serta memberikan respons cepat terhadap kondisi darurat. Dalam mata kuliah Embedded System, salah satu tantangan utama dalam pengembangan sistem adalah bagaimana mengintegrasikan embedded software, embedded hardware, dan physical environment secara menyeluruh dengan memperhatikan aspek dependability, real-time processing, power efficiency, dan reliability. Oleh karena itu, proyek “FIRESENTRY: Mitigasi Kebakaran Dini Berbasis IoT” dikembangkan sebagai implementasi konsep embedded system berbasis Internet of Things (IoT) untuk mendukung keamanan lingkungan.

FIRESENTRY dirancang menggunakan mikrokontroler ESP32 yang terintegrasi dengan sensor Flame IR, MQ-2, dan DS18B20 untuk mendeteksi api, gas/asap, serta suhu secara real-time. Sistem menerapkan metode sensor fusion agar proses pengambilan keputusan menjadi lebih akurat dan mampu meminimalkan false alarm. Sebagai respons lokal, sistem dilengkapi dengan alarm audio frekuensi tinggi melalui buzzer, indikator visual berkode warna menggunakan LED RGB, serta visualisasi data pada layar OLED untuk memberikan informasi kondisi lingkungan secara langsung kepada pengguna. Selain itu, sistem memanfaatkan konektivitas nirkabel untuk mengirimkan notifikasi darurat kritis multi-level ke platform IoT sehingga pengguna dapat melakukan tindakan penanganan lebih cepat.

Dengan adanya sistem ini, diharapkan proses deteksi dan penyebaran informasi mitigasi kebakaran dapat dilakukan secara lebih cepat, efektif, dan andal sehingga mampu mengurangi risiko korban jiwa maupun kerugian material akibat keterlambatan evakuasi dan penanganan kebakaran.

1.2 RUMUSAN MASALAH

1. Bagaimana merancang dan mengimplementasikan sistem *embedded* berbasis ESP32 yang mampu mendeteksi potensi kebakaran secara *real-time* melalui integrasi sensor Flame IR, MQ-2, dan DS18B20?
2. Bagaimana menerapkan metode *sensor fusion* pada sistem FIRESENTRY agar proses pengambilan keputusan kondisi bahaya lebih akurat dan mampu meminimalkan *false alarm*?
3. Bagaimana merancang mekanisme respons mitigasi aktif berupa peringatan audio-visual lokal terintegrasi dan notifikasi darurat multi-level berbasis IoT ketika kondisi bahaya terdeteksi?
4. Bagaimana memastikan sistem FIRESENTRY memenuhi aspek *dependability*, *real-time processing*, dan *reliability* dalam lingkungan *embedded system* yang terintegrasi?

1.3 TUJUAN

1. Merancang dan mengimplementasikan sistem *embedded* berbasis ESP32 yang mengintegrasikan sensor Flame IR, MQ-2, dan DS18B20 untuk mendeteksi api, gas/asap, serta suhu lingkungan secara *real-time*.
2. Menerapkan metode *sensor fusion* pada sistem FIRESENTRY sehingga keputusan kondisi bahaya diambil berdasarkan kombinasi data multisensor yang lebih akurat dan meminimalkan terjadinya *false alarm*.
3. Merancang mekanisme respons mitigasi aktif pada sistem FIRESENTRY yang mencakup visualisasi data lokal (OLED dan LED RGB), alarm suara (*buzzer*), serta notifikasi darurat multi-level berbasis IoT secara langsung ketika kondisi siaga maupun bahaya terdeteksi.
4. Memastikan sistem FIRESENTRY memenuhi aspek *dependability*, *real-time processing*, dan *reliability* sebagai implementasi konsep *embedded system* yang handal dalam mendukung keamanan lingkungan.

1.4 MANFAAT

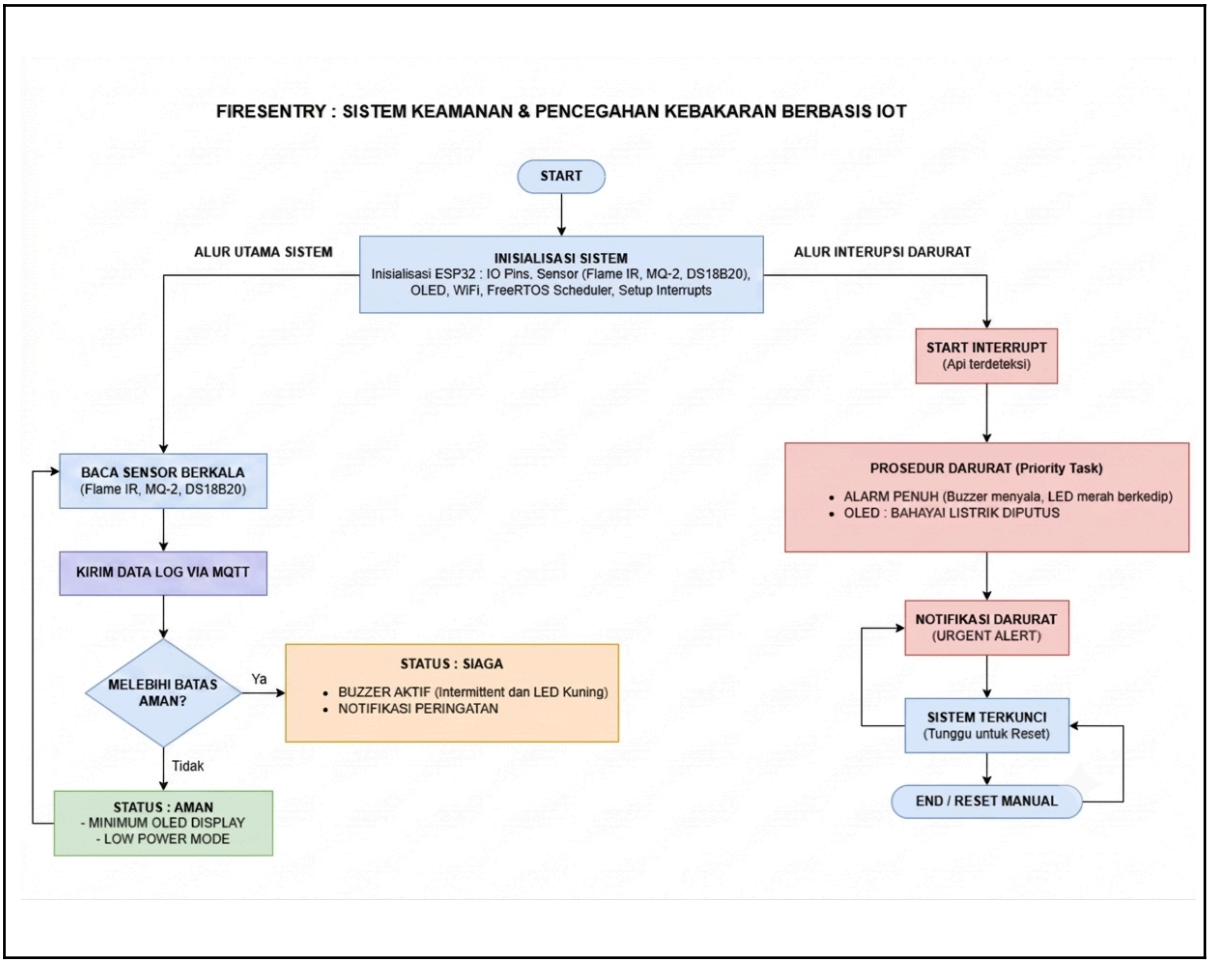
1. Menjadi solusi mitigasi kebakaran dini yang bekerja secara otomatis dan mandiri berbasis teknologi *embedded system* dan IoT, sehingga mampu mendeteksi potensi bahaya sejak fase awal guna meminimalkan risiko jatuhnya korban jiwa.
2. Memberikan lapisan keamanan tambahan melalui mekanisme peringatan audio-visual lokal terintegrasi yang responsif, sehingga dapat memberikan panduan evakuasi instan bagi penghuni gedung sirkuit lokal saat terjadi kondisi darurat.

3. Meningkatkan kecepatan dan efektivitas penanganan darurat melalui notifikasi *real-time* multi-level berbasis IoT yang memungkinkan pengguna mendapatkan informasi detail mengenai parameter bahaya secara langsung meskipun tidak berada di lokasi.
4. Meminimalkan terjadinya *false alarm* dalam proses deteksi kebakaran melalui penerapan metode *sensor fusion* yang mengintegrasikan data dari sensor Flame IR, MQ-2, dan DS18B20 secara bersamaan sehingga sistem memiliki tingkat akurasi tinggi.
5. Memberikan kontribusi akademis dalam bidang *embedded system*, khususnya terkait implementasi integrasi multisensor, *real-time processing*, dan *automatic response* pada sistem keamanan berbasis *cyber-physical systems*.
6. Menjadi referensi bagi pengembang teknologi dalam merancang sistem *embedded* IoT yang menggabungkan mekanisme *decision-making* berbasis *sensor fusion* dengan komunikasi data *real-time* terstruktur pada platform IoT tanpa bergantung pada aktuator mekanis eksternal.

BAB II

PERANCANGAN SISTEM

2.1 DESIGN FLOW



Gambar 1. Design Flow

Sistem keamanan dan pencegahan kebakaran berbasis IoT FIRESENTRY diawali dengan proses inisialisasi sistem yang dilakukan oleh mikrokontroler ESP32 untuk mengaktifkan seluruh komponen, meliputi pin input/output (I/O), sensor Flame IR, MQ-2, DS18B20, layar OLED, koneksi WiFi, MQTT Broker, serta konfigurasi *interrupt* untuk deteksi api. Setelah inisialisasi selesai, sistem terbagi menjadi dua jalur operasi yang berjalan sesuai fungsinya masing-masing, yaitu alur utama sistem dan alur interupsi darurat. Pada alur utama sistem, ESP32 melakukan pembacaan terhadap sensor Flame IR, MQ-2, dan DS18B20 secara berkala untuk memantau kondisi lingkungan, lalu mengirimkan data log tersebut secara langsung melalui protokol MQTT untuk pemantauan jarak jauh.

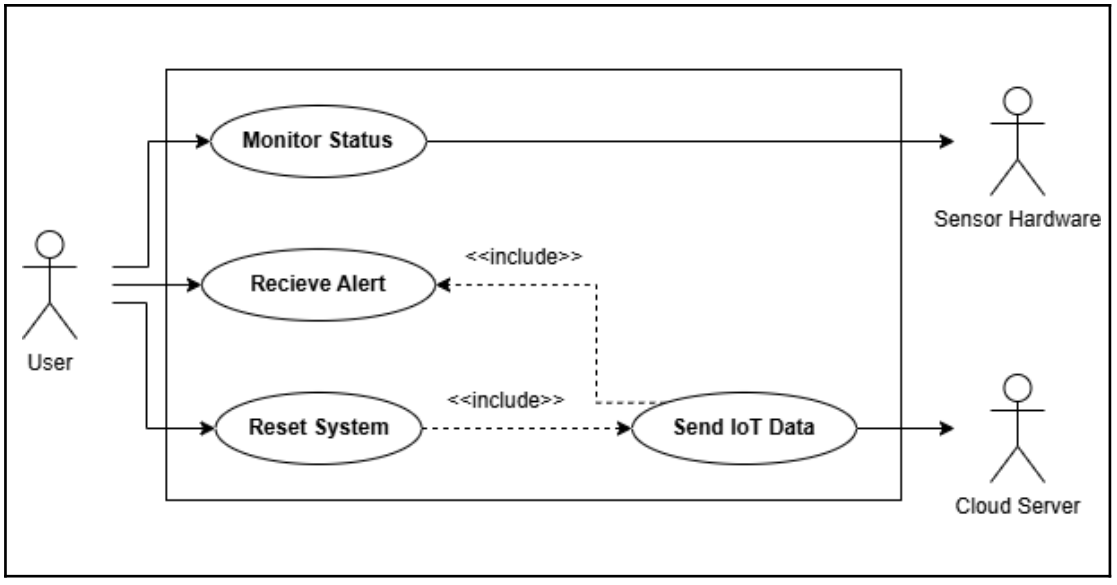
Setelah pengiriman data selesai, sistem mengevaluasi parameter lingkungan pada blok keputusan “Melebihi Batas Aman?”. Jika nilai yang terukur masih berada di bawah ambang batas, sistem masuk ke dalam Status Aman; pada kondisi ini, layar OLED hanya

menampilkan informasi minimum dan sistem beralih ke *Low Power Mode* demi efisiensi daya sebelum akhirnya kembali ke tahap pembacaan sensor secara kontinu. Sebaliknya, jika nilai sensor melebihi batas aman, sistem akan beralih ke Status Siaga yang ditandai dengan aktifnya *buzzer* secara berkala (*intermittent*), menyalanya LED kuning sebagai indikator kewaspadaan, serta pengiriman notifikasi peringatan kepada pengguna, lalu sistem tetap melanjutkan siklus monitoring kembali ke proses pembacaan sensor.

Di sisi lain, FIRESENTRY dilengkapi dengan Alur Interupsi Darurat yang bekerja secara independen dengan prioritas tertinggi, di mana jalur ini akan langsung terpicu (*Start Interrupt*) seketika sensor Flame IR mendeteksi keberadaan api tanpa harus menunggu proses di alur utama selesai. Ketika *interrupt* aktif, sistem langsung mengeksekusi Prosedur Darurat (*Priority Task*) berupa aktivasi alarm penuh, yaitu *buzzer* yang menyala terus-menerus dan LED merah yang berkedip serta menampilkan pesan bahaya pada OLED bahwa aliran listrik telah diputus sebagai langkah mitigasi awal. Setelah prosedur darurat berjalan, sistem akan mengirimkan Notifikasi Darurat (*Urgent Alert*) kepada pengguna dan langsung mengunci diri pada kondisi Sistem Terkunci (Tunggu untuk Reset). Sistem akan terus mempertahankan status darurat ini dan tidak akan kembali ke mode pemantauan normal secara otomatis sampai pengguna melakukan *End/Reset* Manual untuk memulai kembali seluruh operasional sistem dari awal.

2.2 STATECHART DAN UML

2.2.1 USE CASE DIAGRAM



Gambar 2. Use Case Diagram

Pada bagian kiri diagram terdapat aktor *User* yang berinteraksi langsung dengan tiga fungsi utama sistem, yaitu Monitor Status, Receive Alert, dan Reset System. Fungsi Monitor Status digunakan pengguna untuk memantau kondisi lingkungan secara real-time, seperti suhu, kadar gas/asap, dan status deteksi api yang diperoleh dari sensor. Informasi tersebut

ditampilkan melalui media monitoring sehingga pengguna dapat mengetahui kondisi sistem kapan saja.

Fungsi Receive Alert menggambarkan proses ketika pengguna menerima notifikasi peringatan dari sistem. Notifikasi ini akan dikirim apabila sensor mendeteksi adanya peningkatan suhu, gas/asap, atau keberadaan api yang mengindikasikan potensi kebakaran. Dalam implementasi proyek FIRESENTRY, peringatan dikirim melalui bot Telegram sehingga pengguna dapat menerima informasi secara cepat melalui smartphone maupun perangkat lain yang terhubung internet.

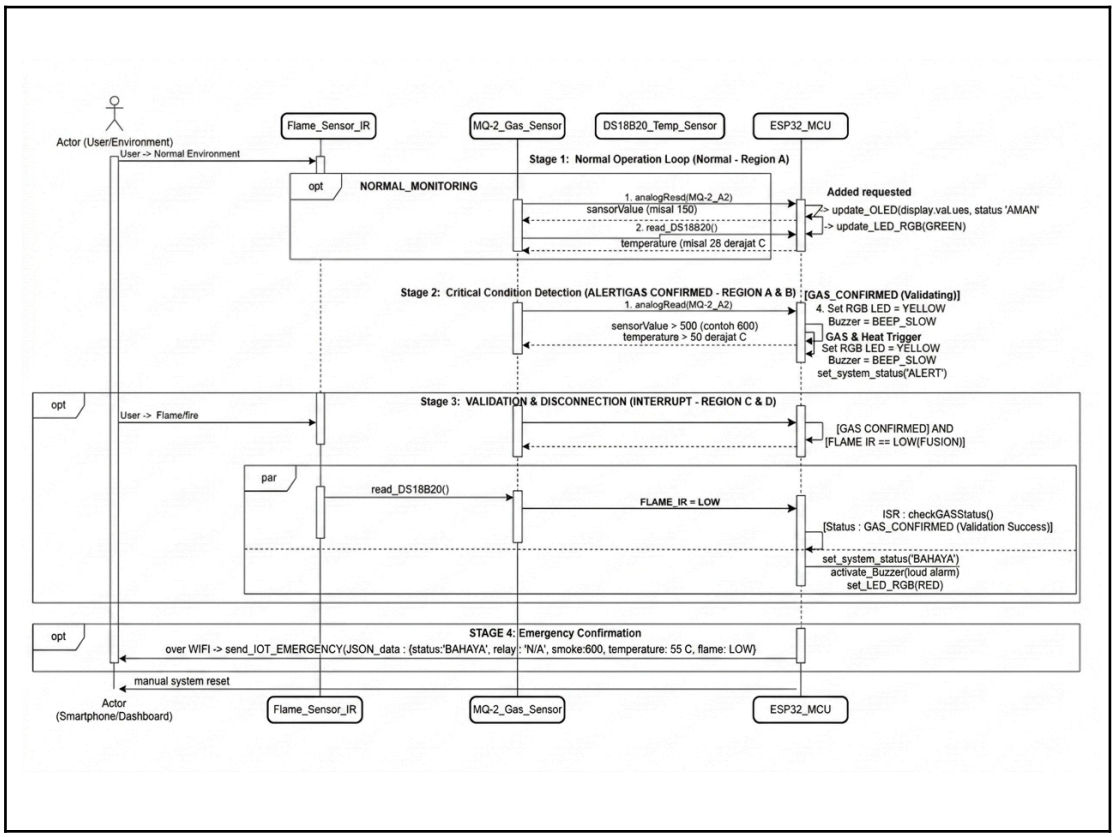
Selanjutnya, fungsi Reset System digunakan untuk mengembalikan sistem ke kondisi normal setelah keadaan dinyatakan aman. Setelah alarm atau kondisi darurat selesai ditangani, pengguna dapat melakukan reset agar sistem kembali menjalankan proses monitoring secara normal.

Pada sisi kanan diagram terdapat aktor Sensor Hardware yang berhubungan dengan fungsi Monitor Status. Aktor ini merepresentasikan perangkat sensor yang digunakan dalam proyek, yaitu DS18B20, MQ-2, dan Flame IR Sensor. Sensor-sensor tersebut berfungsi membaca kondisi fisik lingkungan kemudian mengirimkan data ke ESP32 untuk diproses lebih lanjut.

Diagram juga menunjukkan fungsi Send IoT Data yang terhubung dengan aktor Cloud Server. Fungsi ini menggambarkan proses pengiriman data monitoring dan notifikasi dari ESP32 menuju layanan IoT melalui jaringan internet. Pada proyek FIRESENTRY, cloud server direpresentasikan oleh layanan bot Telegram yang digunakan sebagai media monitoring dan pengiriman notifikasi secara real-time.

Relasi <> pada diagram menunjukkan bahwa fungsi Receive Alert dan Reset System melibatkan proses Send IoT Data. Artinya, ketika sistem mengirimkan notifikasi peringatan kepada pengguna maupun ketika sistem melakukan reset, ESP32 tetap mengirimkan informasi tersebut melalui layanan IoT agar pengguna dapat mengetahui perubahan kondisi sistem secara real-time.

2.2.2 SEQUENCE DIAGRAM



Gambar 3. Sequence Diagram

Pada Stage 1: Normal Operation Loop, sistem berada pada kondisi pemantauan normal. ESP32 secara berkala membaca nilai sensor gas menggunakan fungsi analogRead(MQ-2_A2) dan sensor suhu menggunakan read_DS18B20(). Contoh nilai yang diperoleh adalah sensor gas sekitar 150 dan temperatur sekitar 28°C yang masih dianggap aman. Setelah data dibaca, ESP32 memperbarui tampilan OLED dengan status “AMAN” dan menyalakan LED RGB berwarna hijau sebagai indikator bahwa lingkungan berada dalam kondisi normal. Tahap ini berlangsung secara terus-menerus selama tidak ada indikasi bahaya.

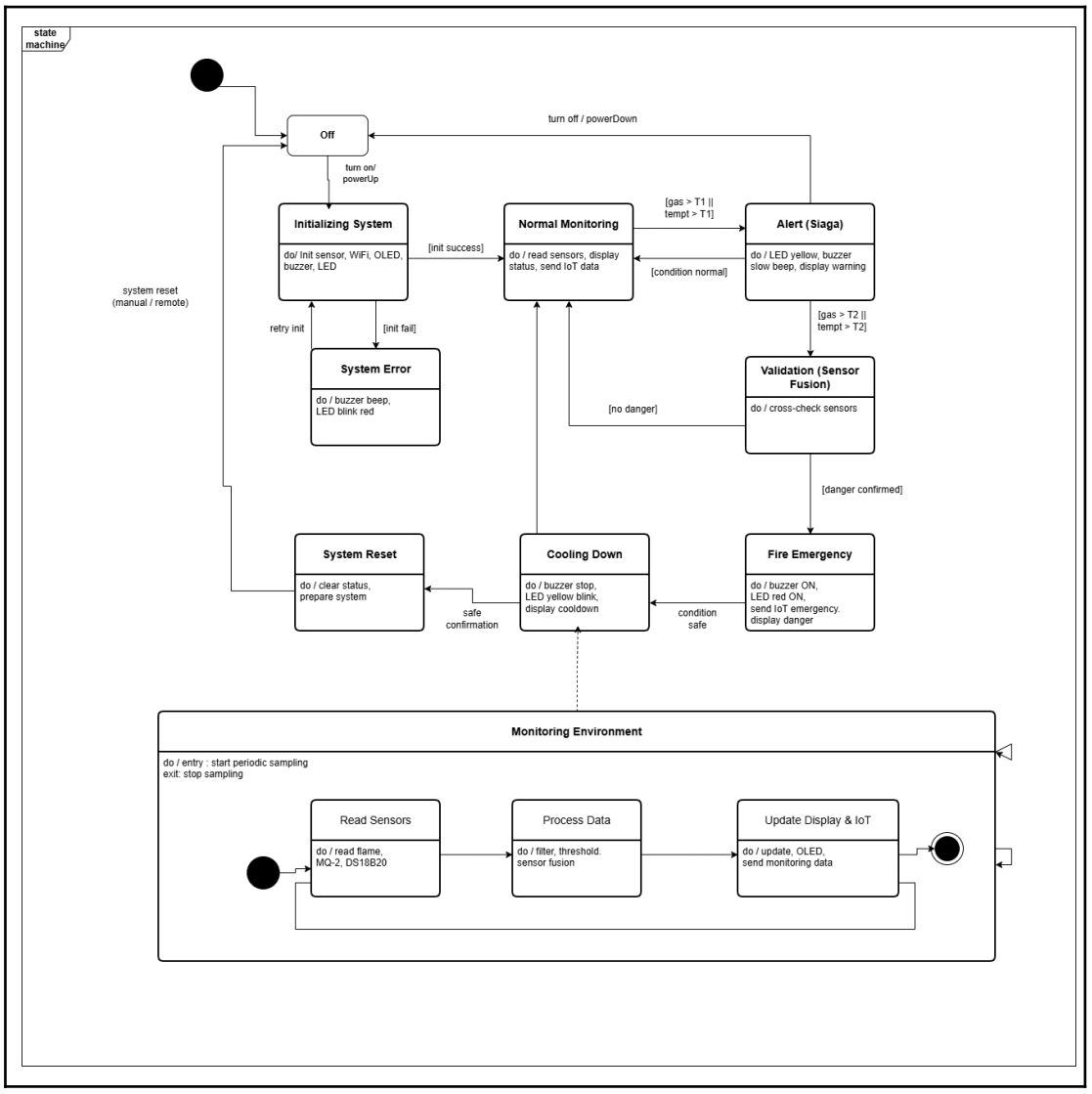
Pada Stage 2: Critical Condition Detection, sistem mulai mendeteksi adanya kondisi abnormal ketika nilai sensor gas meningkat melebihi ambang batas, misalnya lebih dari 500, dan suhu lingkungan melebihi 50°C. Ketika kondisi ini terjadi, ESP32 mengubah status sistem menjadi siaga atau ALERT. Sistem kemudian menyalakan LED kuning, mengaktifkan buzzer dengan bunyi lambat (*BEEP_SLOW*), serta memperbarui status gas dan temperatur tinggi pada tampilan sistem. Tahap ini berfungsi sebagai peringatan awal bahwa terdapat potensi kebakaran atau kebocoran gas.

Selanjutnya pada Stage 3: Validation & Disconnection, sistem melakukan validasi tambahan menggunakan sensor api Flame Sensor IR. Pada bagian ini terdapat mekanisme *parallel process* (par) yang menunjukkan bahwa pembacaan sensor suhu dan sensor api dapat

berlangsung secara bersamaan. Jika sensor api mendeteksi nyala api dengan kondisi FLAME_IR == LOW dan status gas sebelumnya telah terkonfirmasi, maka proses interrupt dijalankan oleh ESP32 melalui ISR checkGASStatus(). Setelah validasi berhasil, sistem menetapkan status menjadi “BAHAYA”, mengaktifkan buzzer darurat, dan mengubah LED RGB menjadi merah sebagai indikator kondisi kebakaran yang telah dipastikan melalui sensor fusion.

Pada Stage 4: Emergency Confirmation, sistem memasuki kondisi darurat penuh. ESP32 mengirimkan data darurat melalui koneksi WiFi ke dashboard atau smartphone pengguna menggunakan format JSON. Informasi yang dikirim meliputi status bahaya, nilai asap/gas, temperatur, dan kondisi sensor api. Contoh data yang dikirim adalah status “BAHAYA”, nilai smoke 600, temperatur 55°C, dan flame LOW. Setelah pengguna menerima notifikasi, sistem dapat di-reset secara manual melalui dashboard atau perangkat kontrol untuk mengembalikan sistem ke kondisi monitoring normal.

2.2.3 STATECHART DIAGRAM



Gambar 4. Statechart Diagram

Proses dimulai dari state Off, yaitu kondisi ketika sistem belum aktif. Saat perangkat dinyalakan (*turn on/powerUp*), sistem berpindah ke state Initializing System untuk melakukan inisialisasi komponen seperti sensor, WiFi, OLED, buzzer, dan LED. Jika proses inisialisasi berhasil, sistem masuk ke state Normal Monitoring untuk membaca sensor, menampilkan status, dan mengirim data ke IoT. Namun apabila inisialisasi gagal, sistem berpindah ke state System Error, di mana buzzer berbunyi dan LED merah berkedip sebagai tanda kesalahan. Setelah dilakukan percobaan ulang (*retry init*), sistem dapat kembali ke proses inisialisasi.

Pada kondisi Normal Monitoring, sistem terus memantau nilai gas dan temperatur lingkungan. Jika nilai sensor melebihi ambang pertama (T1), sistem masuk ke state Alert (Siaga) sebagai peringatan awal. Pada tahap ini LED kuning menyala, buzzer berbunyi pelan, dan sistem menampilkan pesan peringatan. Apabila kondisi kembali normal, sistem akan

kembali ke mode monitoring biasa. Namun jika nilai sensor meningkat hingga melewati ambang kedua (T2), sistem masuk ke state Validation (Sensor Fusion) untuk melakukan validasi data dengan membandingkan hasil beberapa sensor agar mengurangi kemungkinan kesalahan pembacaan.

Setelah proses validasi, apabila bahaya benar-benar terkonfirmasi, sistem berpindah ke state Fire Emergency. Pada kondisi ini buzzer menyala penuh, LED merah aktif, tampilan menunjukkan status bahaya, dan sistem mengirim notifikasi darurat melalui IoT. Ketika kondisi lingkungan mulai aman, sistem masuk ke state Cooling Down, yaitu tahap pendinginan dan penstabilan sistem. Pada tahap ini buzzer dimatikan, LED kuning berkedip, dan sistem menunggu hingga keadaan benar-benar aman. Setelah mendapatkan konfirmasi aman (*safe confirmation*), sistem berpindah ke state System Reset untuk membersihkan status sebelumnya dan mempersiapkan sistem kembali bekerja normal. Setelah reset selesai, sistem kembali ke kondisi monitoring awal atau dapat dimatikan kembali menuju state Off.

Bagian bawah diagram menunjukkan sub-state Monitoring Environment, yang menjelaskan proses internal selama monitoring berlangsung. Ketika state ini aktif, sistem memulai pengambilan sampel sensor secara periodik. Proses dimulai dari Read Sensors untuk membaca sensor flame, MQ-2, dan DS18B20. Data kemudian diproses pada state Process Data dengan melakukan filtering, threshold checking, dan sensor fusion. Setelah itu hasil dipublikasikan pada state Update Display & IoT untuk memperbarui tampilan OLED dan mengirim data monitoring ke platform IoT. Siklus ini berlangsung terus menerus selama sistem berada dalam mode pemantauan lingkungan.

2.3 INTERFACING DENGAN LINGKUNGAN

2.3.1 SENSOR

Sensor Api (Flame IR Sensor)

Flame IR Sensor pada proyek FIRESENTRY bekerja dengan mendeteksi radiasi inframerah yang dihasilkan oleh nyala api pada panjang gelombang sekitar 760 nm hingga 1100 nm. Sensor ini menghasilkan keluaran digital (digital output) berupa kondisi HIGH atau LOW sehingga termasuk dalam kategori input digital. Modul ini dilengkapi komparator LM393 agar sinyal keluaran menjadi lebih stabil dan mudah diproses oleh ESP32. Karena menggunakan sinyal digital, sensor dapat dihubungkan dengan mekanisme hardware interrupt sehingga ketika api terdeteksi, sistem mampu memberikan respons darurat secara instan tanpa harus menunggu proses sampling berkala.

Sensor Gas dan Asap (MQ-2)

Sensor MQ-2 digunakan pada FIRESENTRY untuk membaca konsentrasi gas dan asap di lingkungan. Sensor ini mengubah kandungan gas fisik menjadi perubahan tegangan listrik analog yang dapat diproses oleh mikrokontroler. MQ-2 termasuk kategori input analog karena nilai keluarannya berubah secara kontinu berdasarkan kadar gas yang terdeteksi. Oleh sebab itu, ESP32 memanfaatkan fitur Analog to Digital Converter (ADC) untuk mengubah sinyal analog tersebut menjadi data digital sebelum diproses lebih lanjut. Pembacaan sensor dilakukan secara berkala melalui proses sampling agar sistem mampu memantau perubahan kadar asap dan gas secara real-time. Data yang diperoleh kemudian difilter menggunakan threshold tertentu dan dikombinasikan dengan sensor lain melalui metode sensor fusion untuk mengurangi kemungkinan false alarm.

Sensor Suhu (DS18B20)

DS18B20 digunakan dalam FIRESENTRY untuk memantau perubahan temperatur lingkungan sebagai indikator awal potensi kebakaran. Sensor ini menangkap besaran fisik berupa suhu lalu mengubahnya menjadi data digital yang dapat diproses oleh sistem embedded. Berbeda dengan sensor analog, DS18B20 menggunakan protokol komunikasi digital OneWire sehingga data suhu dapat dikirim langsung ke ESP32 tanpa memerlukan proses ADC tambahan. Sensor ini memiliki rentang pengukuran suhu yang luas dan mampu memberikan data dengan tingkat kestabilan yang baik karena lebih tahan terhadap gangguan elektromagnetik.

Dalam implementasinya, pembacaan suhu dari DS18B20 dikirimkan secara periodik oleh ESP32 melalui protokol MQTT ke broker untuk kemudian diproses dan dipantau oleh sistem secara real-time. Dengan mekanisme ini, data suhu dapat diakses secara terpusat dan konsisten melalui jalur komunikasi IoT, sehingga perubahan temperatur abnormal dapat segera terdeteksi sebagai bagian dari sistem mitigasi kebakaran dini.

2.3.2 INPUT DIGITAL DAN ANALOG

Input digital diterapkan pada penggunaan Flame IR Sensor dan sensor suhu DS18B20. Flame IR Sensor menghasilkan dua kondisi logika, yaitu HIGH ketika tidak ada api dan LOW ketika api terdeteksi. Karena hanya memiliki dua status, sensor ini termasuk kategori input digital dan sangat cocok digunakan pada mekanisme hardware interrupt agar ESP32 dapat memberikan respons secara instan saat api muncul. Selain itu, DS18B20 juga menggunakan komunikasi digital melalui protokol OneWire, sehingga data suhu dikirim langsung dalam bentuk data digital ke mikrokontroler tanpa memerlukan proses konversi tambahan. Penggunaan input digital membuat proses pembacaan data menjadi lebih stabil, cepat, dan minim gangguan noise.

Input analog pada FIRESENTRY diterapkan pada sensor MQ-2 yang digunakan untuk mendeteksi gas dan asap. Sensor ini menghasilkan tegangan analog yang nilainya berubah secara kontinu sesuai konsentrasi gas atau asap di udara. Semakin tinggi kadar gas yang terdeteksi, maka semakin besar nilai tegangan keluarannya. Karena mikrokontroler hanya dapat memproses data digital, ESP32 menggunakan fitur Analog to Digital Converter (ADC) untuk mengubah sinyal analog tersebut menjadi data digital sebelum diproses lebih lanjut. Data hasil pembacaan kemudian dianalisis menggunakan threshold tertentu dan dikombinasikan dengan sensor lain melalui metode sensor fusion agar sistem mampu mendeteksi potensi kebakaran secara lebih akurat dan mengurangi kemungkinan false alarm.

2.3.3 SIGNAL

Pada sistem FIRESENTRY, signal merupakan sinyal listrik yang dihasilkan oleh sensor setelah membaca kondisi fisik lingkungan, seperti suhu, gas/asap, dan keberadaan api. Sinyal tersebut kemudian dikirim ke mikrokontroler ESP32 untuk diproses menjadi informasi kondisi lingkungan secara real-time. Dalam embedded system, signal didefinisikan sebagai pemetaan dari domain waktu (*time domain*) ke domain nilai (*value domain*), sehingga nilai sinyal dapat berubah terhadap waktu sesuai perubahan kondisi lingkungan yang terdeteksi sensor.

Pada Flame IR Sensor, signal yang dihasilkan berupa sinyal digital dengan dua kondisi logika, yaitu HIGH dan LOW. Ketika tidak terdapat api, sensor menghasilkan logika HIGH, sedangkan saat api terdeteksi sinyal berubah menjadi LOW. Karena sinyal bersifat digital dan diskrit, perubahan kondisi dapat langsung diproses oleh ESP32 menggunakan mekanisme *hardware interrupt*. Dengan metode tersebut, sistem mampu memberikan respons secara cepat ketika terjadi indikasi kebakaran tanpa harus menunggu proses pembacaan sensor berikutnya.

Sementara itu, sensor MQ-2 menghasilkan signal analog berupa perubahan tegangan listrik yang nilainya berubah secara kontinu sesuai konsentrasi gas atau asap di udara. Semakin tinggi kadar gas atau asap yang terdeteksi, maka semakin besar nilai tegangan keluaran sensor. Signal analog tersebut dibaca secara berkala melalui proses *sampling*

menggunakan fitur Analog to Digital Converter (ADC) pada ESP32 agar dapat diubah menjadi data digital sebelum diproses lebih lanjut oleh sistem.

Pada sensor DS18B20, signal yang dihasilkan berupa data digital serial menggunakan protokol komunikasi OneWire. Sensor ini membaca temperatur lingkungan kemudian mengirimkan nilai suhu dalam bentuk data digital langsung ke ESP32. Penggunaan komunikasi digital membuat sinyal yang diterima mikrokontroler menjadi lebih stabil dan tidak mudah terpengaruh noise atau gangguan elektromagnetik dibandingkan sinyal analog.

Seluruh signal dari ketiga sensor tersebut diproses secara bersamaan menggunakan metode sensor fusion untuk menentukan kondisi lingkungan. Berdasarkan hasil pemrosesan signal, sistem dapat mengidentifikasi status aman, siaga, atau bahaya. Apabila terdeteksi adanya potensi kebakaran, ESP32 akan mengaktifkan actuator seperti buzzer, LED indikator, tampilan OLED, serta mengirimkan notifikasi IoT kepada pengguna sebagai bentuk respons terhadap perubahan signal yang diterima dari lingkungan.

2.3.4 SAMPLING DAN ALIASING

Proses sampling merupakan tahap pengambilan nilai dari sinyal kontinu pada interval waktu tertentu untuk diproses secara digital. Pada sistem FIRESENTRY, sampling diterapkan pada sensor MQ-2 yang menghasilkan sinyal analog. Berdasarkan teorema sampling Nyquist-Shannon, frekuensi sampling (fs) harus bernilai minimal dua kali frekuensi tertinggi dari sinyal yang diukur ($fs \geq 2 \times fmax$) agar sinyal dapat direkonstruksi secara akurat tanpa kehilangan informasi.

Fenomena aliasing terjadi apabila frekuensi sampling lebih rendah dari syarat Nyquist, sehingga komponen frekuensi tinggi dari sinyal akan terlipat (fold) ke frekuensi yang lebih rendah dan menyebabkan distorsi pada hasil pembacaan data. Pada FIRESENTRY, risiko aliasing diminimalkan karena perubahan kadar gas dan suhu di lingkungan bersifat lambat (low-frequency signal), sehingga interval sampling periodik yang diterapkan sudah mencukupi syarat Nyquist dengan margin yang memadai.

Dalam implementasinya, hasil pembacaan sensor MQ-2 dikirimkan secara berkala oleh ESP32 melalui protokol MQTT ke broker untuk proses monitoring dan analisis data secara real-time. Mekanisme pengiriman data berbasis MQTT ini memungkinkan data sensor tetap terstruktur dan tersinkronisasi dengan sistem pemantauan, sehingga proses pengambilan keputusan dapat dilakukan secara konsisten tanpa mengganggu alur akuisisi data dari sensor.

2.3.5 ADC (Analog to Digital Converter) dan Resolusi

ESP32 memiliki ADC internal dengan resolusi 12-bit, artinya sinyal analog dari sensor MQ-2 dikonversi menjadi nilai diskrit dalam rentang 0 hingga 4095 ($2^{12} = 4096$ nilai). Tegangan referensi ADC ESP32 adalah 3,3V, sehingga resolusi tegangan per step diperoleh dari:

Resolusi tegangan = 3,3V / 4096 ≈ 0,0008 V per step (±0,8 mV)

Semakin tinggi resolusi ADC, semakin halus diskritisasi sinyal analog dan semakin akurat representasi digitalnya. Nilai ADC yang terbaca kemudian dipetakan ke level ambang batas (threshold) untuk menentukan kondisi siaga maupun bahaya. Pada kode sistem FIRESENTRY, threshold gas ditetapkan pada nilai ADC 3950 untuk kondisi SIAGA dan 4090 untuk kondisi BAHAYA.

Kuantisasi adalah proses pembulatan nilai analog ke nilai diskrit ADC terdekat, yang secara inheren menghasilkan quantization error maksimum sebesar ±½ step. Pada resolusi 12-bit, error kuantisasi ini sangat kecil dan tidak berdampak signifikan terhadap akurasi deteksi kebakaran.

2.3.6 FILTERING

Filtering diterapkan untuk menyaring noise dan pembacaan sensor yang tidak stabil sebelum data diproses lebih lanjut. Pada FIRESENTRY terdapat dua lapisan filtering, yaitu threshold dan sensor fusion. Threshold filtering merupakan metode paling sederhana, di mana nilai pembacaan sensor baru dianggap signifikan apabila melampaui nilai ambang batas yang telah ditetapkan. Nilai di bawah threshold diabaikan sehingga variasi kecil akibat noise elektronik tidak memicu false alarm. Sensor fusion sebagai filtering kontekstual merupakan lapisan filtering yang lebih tinggi. Sistem tidak memutuskan status bahaya berdasarkan satu sensor saja, melainkan membutuhkan konfirmasi dari kombinasi beberapa sensor. Misalnya, kondisi BAHAYA hanya dipicu apabila Flame IR mendeteksi api (LOW) sekaligus nilai MQ-2 atau DS18B20 melampaui threshold bahaya. Pendekatan ini secara efektif menyaring false alarm yang berasal dari gangguan satu sensor secara parsial. Dan penggunaan protokol digital OneWire pada DS18B20 secara inherent memberikan efek filtering terhadap noise elektromagnetik, karena data dikirim dalam format digital yang tidak terpengaruh oleh fluktuasi tegangan analog.

2.4 PROCESSING HARDWARE

2.4.1 ALAT DAN BAHAN

1. ESP32 DevKit

ESP32 digunakan sebagai mikrokontroler utama pada sistem FIRESENTRY. Perangkat ini berfungsi sebagai pusat pemrosesan data dari seluruh sensor yang terhubung pada sistem. ESP32 dipilih karena memiliki performa pemrosesan yang tinggi, fitur WiFi bawaan untuk komunikasi IoT, serta dukungan komunikasi berbasis MQTT untuk integrasi dengan sistem broker IoT.

Pada proyek ini, ESP32 bertugas membaca data dari sensor MQ-2, DS18B20, dan Flame IR Sensor, kemudian memproses data tersebut menggunakan metode sensor fusion untuk menentukan kondisi aman, siaga, atau bahaya. Data hasil pemrosesan kemudian dikirimkan melalui protokol MQTT ke broker agar dapat dipantau secara real-time. Selain

itu, ESP32 juga mengendalikan output seperti buzzer, LED indikator, dan OLED display, serta mengirimkan notifikasi ke smartphone pengguna melalui sistem yang terhubung dengan MQTT broker.

2. Sensor Api (Flame IR Sensor)

Flame IR Sensor digunakan untuk mendeteksi keberadaan nyala api secara langsung. Sensor ini bekerja dengan menangkap radiasi inframerah yang dipancarkan api pada panjang gelombang tertentu. Modul sensor menghasilkan output digital berupa HIGH atau LOW sehingga dapat digunakan pada mekanisme *hardware interrupt* untuk memberikan respons cepat ketika api terdeteksi. Dalam sistem FIRESENTRY, flame sensor menjadi komponen penting untuk mendeteksi kondisi kebakaran nyata secara instan.

3. Sensor Gas dan Asap (MQ-2)

Sensor MQ-2 digunakan untuk mendeteksi keberadaan gas mudah terbakar dan asap di lingkungan sekitar. Sensor ini mampu mendeteksi berbagai jenis gas seperti LPG, propana, hidrogen, serta asap hasil pembakaran. MQ-2 menghasilkan sinyal analog berupa perubahan tegangan sesuai konsentrasi gas yang terdeteksi. Data analog tersebut kemudian dibaca oleh ADC internal ESP32 untuk diproses lebih lanjut. Sensor ini berfungsi sebagai sistem deteksi dini sebelum muncul api yang lebih besar.

4. Sensor Suhu (DS18B20)

DS18B20 digunakan untuk memantau temperatur lingkungan secara real-time. Sensor ini memiliki komunikasi digital OneWire sehingga data suhu dapat dikirim langsung ke ESP32 dengan kestabilan yang baik dan minim gangguan noise. Pada proyek FIRESENTRY, sensor suhu digunakan untuk mendeteksi kenaikan temperatur abnormal yang berpotensi menjadi indikasi awal terjadinya kebakaran.

5. OLED Display

OLED display digunakan sebagai media tampilan informasi sistem secara langsung. Layar ini menampilkan data suhu, kadar gas/asap, status kondisi lingkungan, serta pesan peringatan ketika sistem mendeteksi bahaya. Penggunaan OLED memudahkan pengguna dalam melakukan monitoring kondisi sistem tanpa harus membuka dashboard IoT.

6. Buzzer

Buzzer berfungsi sebagai actuator output berupa alarm suara. Pada kondisi siaga, buzzer akan berbunyi secara terputus-putus sebagai peringatan awal. Sementara pada kondisi bahaya atau ketika api terdeteksi, buzzer akan berbunyi terus-menerus untuk memberikan tanda evakuasi kepada pengguna. Komponen ini berperan penting dalam memberikan notifikasi audio secara langsung.

7. LED Indikator

LED indikator digunakan sebagai penanda visual kondisi sistem. Pada proyek FIRESENTRY digunakan beberapa LED dengan fungsi berbeda, seperti LED kuning untuk menunjukkan status siaga dan LED merah untuk menunjukkan kondisi bahaya. Penggunaan

LED memudahkan pengguna mengetahui kondisi sistem secara cepat hanya melalui indikator cahaya.

8. Kabel Jumper

Kabel jumper digunakan sebagai media penghubung antar komponen elektronik pada breadboard maupun ESP32. Kabel ini menghubungkan sensor, OLED, buzzer, dan LED ke pin input/output mikrokontroler sehingga seluruh komponen dapat saling berkomunikasi dalam sistem.

9. Breadboard

Breadboard digunakan sebagai media perakitan rangkaian sementara tanpa perlu melakukan penyolderan langsung. Penggunaan breadboard mempermudah proses pengembangan, pengujian, dan debugging sistem FIRESENTRY karena komponen dapat dipasang dan dilepas dengan mudah.

10. Resistor

Resistor digunakan untuk membatasi arus listrik pada beberapa komponen seperti LED indikator agar tidak mengalami kerusakan akibat arus berlebih. Selain itu, resistor juga digunakan sebagai pull-up resistor pada komunikasi OneWire sensor DS18B20 agar proses komunikasi data berjalan stabil.

11. Kabel USB

Kabel USB digunakan sebagai sumber daya utama untuk menyalakan ESP32 dan seluruh komponen pada sistem. Tegangan yang stabil sangat penting agar pembacaan sensor dan proses komunikasi data dapat berjalan dengan baik tanpa gangguan.

12. Bot Telegram sebagai Dashboard Monitoring IoT

Bot Telegram digunakan sebagai media monitoring dan notifikasi IoT secara real-time yang menghubungkan sistem embedded dengan pengguna melalui jaringan internet. ESP32 akan mengirimkan data hasil pembacaan sensor, seperti suhu lingkungan, kadar gas dan asap, serta status deteksi api ke bot Telegram melalui koneksi WiFi. Informasi tersebut kemudian diterima pengguna dalam bentuk pesan otomatis yang dapat diakses melalui smartphone maupun komputer. Selain digunakan untuk monitoring kondisi lingkungan, bot Telegram juga berfungsi sebagai sistem peringatan dini, di mana sistem akan mengirimkan notifikasi siaga ketika terjadi peningkatan suhu atau gas, serta mengirimkan notifikasi darurat (*urgent alert*) ketika api terdeteksi. Penggunaan bot Telegram membuat proses monitoring menjadi lebih praktis, cepat, dan efisien tanpa memerlukan aplikasi khusus tambahan sehingga pengguna dapat memantau kondisi sistem FIRESENTRY dari jarak jauh secara real-time.

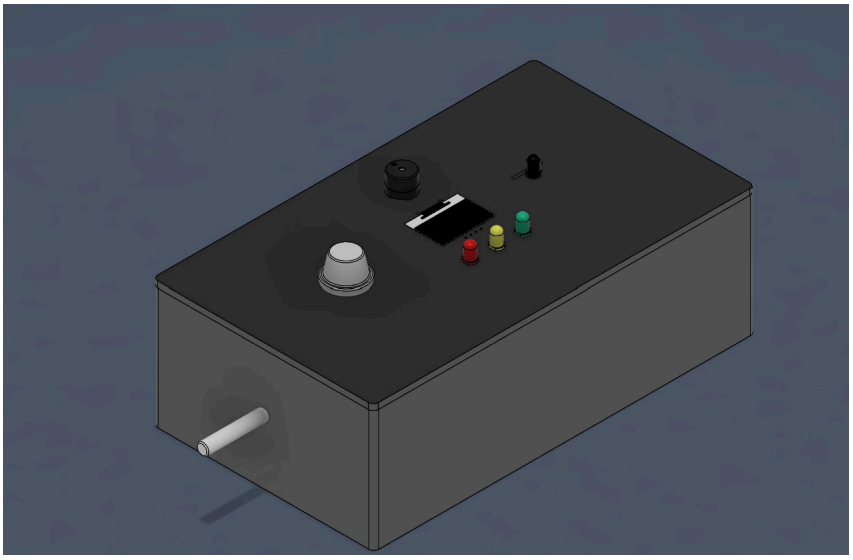
13. Software Arduino IDE

Arduino IDE digunakan sebagai perangkat lunak untuk menulis, mengedit, mengompilasi, dan mengunggah program ke ESP32. Melalui software ini, seluruh logika sistem FIRESENTRY dikembangkan, termasuk pembacaan sensor, pengolahan data, komunikasi IoT, dan pengendalian actuator.

14. MQTT Broker

MQTT Broker digunakan sebagai pusat komunikasi data pada sistem FIRESENTRY untuk mendukung proses monitoring berbasis Internet of Things (IoT). Dalam proyek ini, ESP32 berperan sebagai publisher yang mengirimkan data hasil pembacaan sensor gas MQ-2, sensor suhu DS18B20, dan flame sensor ke MQTT Broker melalui jaringan WiFi. Broker kemudian meneruskan data tersebut kepada subscriber, seperti dashboard monitoring atau aplikasi pengguna, yang berlangganan pada topik tertentu. Penggunaan MQTT Broker memungkinkan proses pertukaran data berlangsung secara real-time, efisien, dan andal karena perangkat pengirim dan penerima tidak perlu terhubung secara langsung. Selain itu, mekanisme publish-subscribe yang diterapkan MQTT memudahkan pengembangan sistem monitoring jarak jauh serta mendukung pengiriman data sensor dan notifikasi kondisi bahaya secara cepat dan terpusat.

2.4.2 DESAIN SISTEM



Gambar 5. Desain Sistem

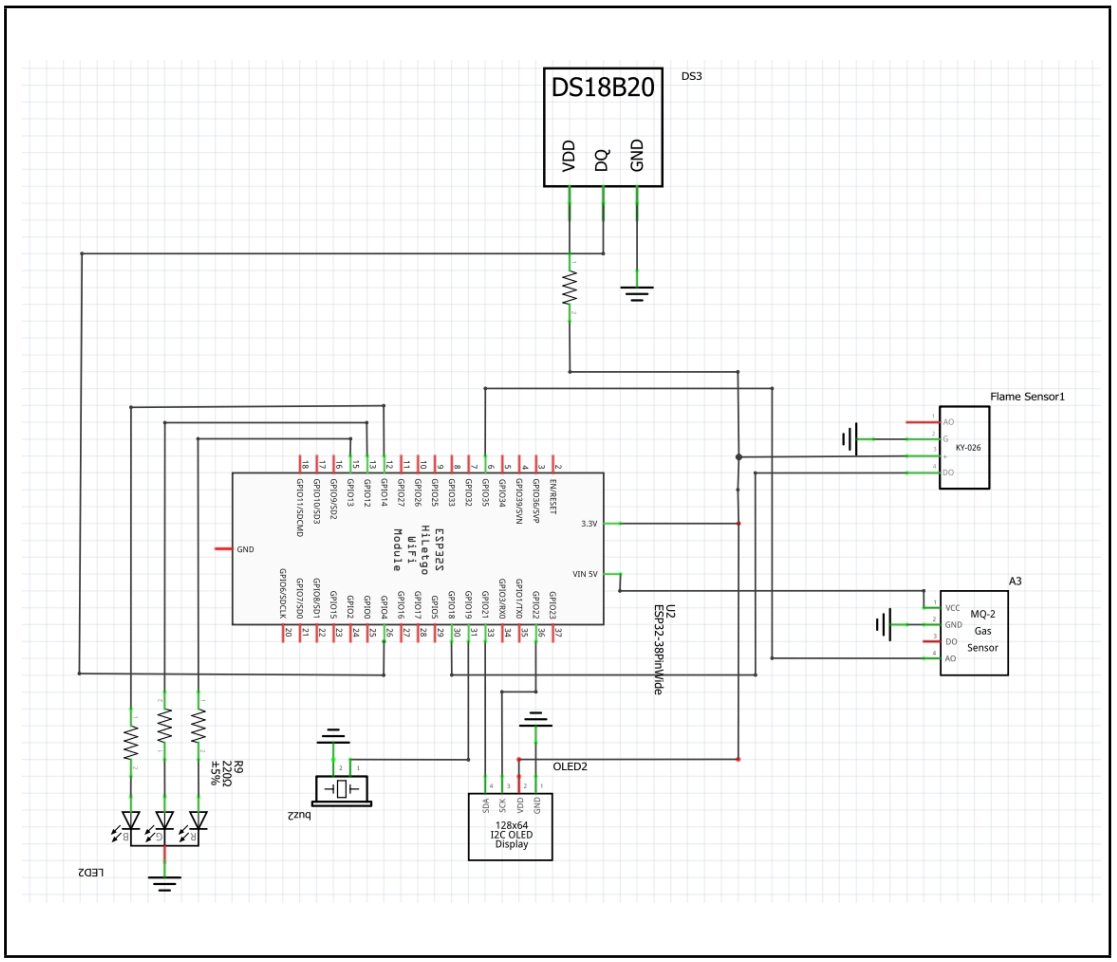
Pada bagian samping enclosure terdapat sensor DS18B20 yang menonjol keluar dari kotak. Sensor ini berfungsi untuk mengukur suhu lingkungan secara real-time. Penempatan sensor di luar enclosure bertujuan agar sensor dapat menerima paparan suhu lingkungan secara langsung sehingga hasil pengukuran menjadi lebih akurat dan tidak terpengaruh oleh suhu komponen elektronik di dalam kotak. Data suhu yang diperoleh kemudian digunakan sebagai salah satu parameter dalam menentukan kondisi aman, waspada, atau bahaya.

Di bagian atas enclosure terdapat sensor MQ-2 yang ditunjukkan oleh komponen berbentuk silinder berwarna putih. Sensor ini berfungsi untuk mendeteksi keberadaan asap maupun gas yang berpotensi menjadi indikasi awal terjadinya kebakaran. Selain itu, terdapat IR Flame Sensor yang berada di dekat layar OLED dan berfungsi untuk mendeteksi radiasi inframerah yang dipancarkan oleh nyala api. Sensor ini memungkinkan sistem mendeteksi keberadaan api secara langsung sehingga dapat memberikan peringatan lebih cepat ketika terjadi kebakaran.

Pada bagian tengah terdapat layar OLED 0,96 inci yang digunakan sebagai media tampilan informasi sistem. Layar ini menampilkan nilai suhu yang terukur, status deteksi sensor, serta kondisi sistem seperti AMAN, ALERT, atau BAHAYA. Dengan adanya layar OLED, pengguna dapat memantau kondisi lingkungan secara langsung tanpa harus mengakses dashboard monitoring.

Sebagai indikator visual, sistem dilengkapi dengan tiga buah LED yang terdiri dari LED merah, LED kuning, dan LED hijau. LED hijau menunjukkan bahwa kondisi lingkungan berada dalam keadaan aman dan tidak terdeteksi adanya indikasi kebakaran. LED kuning menunjukkan kondisi waspada (alert), yaitu ketika terjadi peningkatan parameter tertentu seperti suhu atau konsentrasi gas yang mulai mendekati batas bahaya. Sementara itu, LED merah akan menyala ketika sistem mendeteksi kondisi berbahaya, seperti terdeteksinya api secara langsung atau parameter sensor telah melewati ambang batas yang ditentukan. Kombinasi ketiga LED tersebut memudahkan pengguna untuk mengetahui kondisi sistem secara cepat melalui indikator visual yang sederhana dan mudah dipahami.

2.4.3 SKEMATIK HARDWARE



Gambar 6. Skematik Hardware

Pada sistem FIRESENTRY, ESP32 DevKit digunakan sebagai mikrokontroler utama yang berfungsi sebagai pusat pemrosesan data dari seluruh sensor dan pengendali aktuator. ESP32 memiliki kemampuan konektivitas WiFi yang memungkinkan integrasi sistem dengan

konsep Internet of Things (IoT), khususnya melalui komunikasi berbasis MQTT (Message Queuing Telemetry Transport). Melalui MQTT broker, seluruh data hasil pembacaan sensor dapat dikirimkan secara terstruktur untuk kebutuhan monitoring dan notifikasi secara real-time.

Selain memproses data sensor, ESP32 juga berfungsi untuk mengendalikan perangkat output seperti OLED display, LED indikator, dan buzzer sebagai media informasi dan sistem peringatan. Seluruh status sistem, seperti kondisi aman, siaga, dan bahaya, ditampilkan secara langsung pada perangkat output tersebut. Selain itu, melalui koneksi WiFi, ESP32 mengirimkan data monitoring serta notifikasi otomatis ke sistem IoT yang terhubung dengan MQTT broker, sehingga pengguna dapat memantau kondisi lingkungan dari jarak jauh secara real-time.

Sensor suhu DS18B20 terhubung ke ESP32 menggunakan tiga pin utama, yaitu VDD, DQ, dan GND. Pin VDD terhubung ke sumber tegangan, pin GND terhubung ke ground, sedangkan pin DQ digunakan sebagai jalur komunikasi digital OneWire menuju ESP32. Pada jalur data tersebut dipasang resistor pull-up yang berfungsi menjaga kestabilan sinyal komunikasi sehingga data suhu dapat dibaca secara akurat dan stabil.

Sensor api Flame IR Sensor terhubung ke ESP32 melalui pin VCC, GND, dan DO (Digital Output). Pin DO dihubungkan ke salah satu pin digital ESP32 untuk menghasilkan sinyal HIGH atau LOW berdasarkan keberadaan api. Ketika api terdeteksi, sensor akan mengirimkan sinyal digital yang kemudian diproses oleh ESP32 untuk mengaktifkan sistem peringatan darurat secara cepat.

Sensor MQ-2 digunakan sebagai sensor gas dan asap yang menghasilkan keluaran sinyal analog. Pin VCC dan GND digunakan sebagai sumber daya, sedangkan pin AO (Analog Output) dihubungkan ke pin ADC pada ESP32. Perubahan konsentrasi gas atau asap di lingkungan akan menghasilkan variasi tegangan analog yang kemudian dibaca oleh ESP32 dan dikonversi menjadi data digital untuk diproses lebih lanjut sebelum dikirim ke MQTT broker.

OLED display 128x64 berbasis I2C digunakan sebagai media tampilan informasi sistem secara real-time. Modul ini terhubung ke ESP32 melalui jalur VCC, GND, SDA, dan SCL. Komunikasi I2C memungkinkan ESP32 menampilkan data suhu, kadar gas, serta status sistem seperti aman, siaga, atau bahaya secara langsung pada layar OLED.

Pada bagian output visual, terdapat LED indikator yang dihubungkan ke pin digital ESP32 melalui resistor pembatas arus sebesar 220Ω. Resistor ini berfungsi untuk membatasi arus agar LED tidak rusak akibat kelebihan daya. LED digunakan sebagai indikator kondisi sistem, misalnya untuk menunjukkan status siaga atau bahaya ketika terdeteksi potensi kebakaran.

Selain itu, rangkaian juga menggunakan buzzer sebagai aktuator audio yang terhubung ke salah satu pin output ESP32. Buzzer berfungsi memberikan alarm suara ketika

sistem memasuki kondisi siaga atau bahaya. Seluruh data hasil pembacaan sensor diproses oleh ESP32, kemudian dikirimkan melalui WiFi menggunakan protokol MQTT ke broker sebagai pusat komunikasi data IoT. Dari broker tersebut, data dapat diteruskan ke aplikasi monitoring maupun sistem notifikasi seperti Telegram untuk memberikan informasi secara real-time kepada pengguna.

2.5 KOMUNIKASI DATA

Sistem FIRESENTRY mengimplementasikan arsitektur komunikasi data yang memadukan transmisi nirkabel (*wireless*) jarak jauh dengan jaringan bus kabel lokal untuk menjamin keandalan pemantauan parameter lingkungan. Berdasarkan topologi dan jangkauannya, sistem ini memisahkan alur pertukaran informasi menjadi dua kategori utama, yakni komunikasi eksternal berbasis IoT (*Internet of Things*) menuju jaringan publik, serta komunikasi data internal antar-komponen elektronik (*hardware*) dalam lingkup sistem embedded lokal.

2.5.1 KOMUNIKASI IOT (CLOUD & TELEGRAM BOT)

Pada sub-sistem komunikasi eksternal, FIRESENTRY memanfaatkan modul transiver WiFi terintegrasi pada mikrokontroler ESP32 sebagai gerbang utama (*gateway*) untuk mentransmisikan data dari titik lokal (*point-to-point*) menuju server internet melalui media udara (*wireless channel*). Dalam kondisi normal atau aman, sistem menerapkan manajemen komunikasi yang efisien untuk meminimalkan beban trafik jaringan. ESP32 secara periodik melakukan *sampling* data dari sensor, mengemasnya menjadi data log (catatan riwayat), lalu mengirimkannya ke *Dashboard* IoT di *cloud*. Pada fase aman ini, Telegram Bot dikonfigurasi dalam mode pasif untuk menghindari banjir data (*redundant alert*) yang tidak perlu pada perangkat seluler pengguna. Kendati demikian, pengguna tetap dapat melakukan panggilan data secara interaktif menggunakan mekanisme permintaan-tanggapan (*request-response*) dengan mengirimkan perintah "/status". Ketika perintah digital tersebut masuk ke sistem, ESP32 akan langsung memproses interupsi perangkat lunak tersebut dan membalasnya dengan mengirimkan rangkuman data parameter sensor terkini.

Sebaliknya, alur komunikasi nirkabel akan otomatis beralih menjadi transmisi prioritas darurat (*emergency state*) dengan latensi rendah apabila logika *sensor fusion* lokal mendeteksi anomali pada lingkungan. Sistem membagi tingkatan respons asinkron ini menjadi dua kondisi kritis. Pertama, status SIAGA atau WASPADA yang ditandai oleh indikator LED kuning, terjadi saat konsentrasi gas atau suhu melampaui batas ambang (*threshold*) normal. Pada kondisi ini, ESP32 bertindak aktif melakukan *push notification* berupa pesan peringatan dini secara otomatis ke Telegram pengguna. Kedua, status BAHAYA dengan indikator kedipan cepat LED merah serta aktivasi alarm suara *buzzer* secara kontinu, yang dipicu secara seketika melalui jalur interupsi perangkat keras (*hardware interrupt*) saat sensor *Flame IR* mendeteksi komponen radiasi api nyata, sehingga mampu memotong antrean proses pengiriman data reguler.

Pada kedua kondisi kritis tersebut, ESP32 bertindak sebagai pemaket data yang mengubah data mentah sensor menjadi format pertukaran data digital terstruktur berbobot ringan, yaitu JSON (*JavaScript Object Notation*) dengan nama variabel IOT_EMERGENCY_JSON_data. Paket data JSON ini membawa muatan informasi (*payload*) yang sangat kritis dan terperinci, meliputi kode level status bahaya sistem (Siaga atau Bahaya), tingkat kepekatan gas/asap dalam satuan ppm, nilai temperatur aktual, serta bit indikator deteksi api. Paket data terstruktur ini kemudian ditransmisikan secara *real-time* melewati tumpukan protokol jaringan WiFi menuju Telegram Bot API. Penerapan format JSON yang efisien ini memastikan pesan peringatan darurat memiliki *throughput* yang optimal dan dapat diterima oleh ponsel pengguna dalam waktu singkat demi percepatan evakuasi dan mitigasi kebakaran, meskipun pengguna berada di luar jangkauan lokasi fisik alat.

2.5.2 PROTOKOL KOMUNIKASI HARDWARE

Di samping mengelola komunikasi nirkabel jarak jauh, mikrokontroler ESP32 bertindak sebagai unit pemroses utama yang mengendalikan interkoneksi bus lokal kabel secara intensif guna mensinkronisasikan data antar-komponen *hardware*. Protokol internal yang pertama adalah I2C (*Inter-Integrated Circuit*), yang dikonfigurasi dalam arsitektur tunggal *Master-Slave*, di mana ESP32 bertindak sebagai *Master* yang mengendalikan penuh inisiasi transfer data, dan layar OLED 0,96 inci bertindak sebagai *Slave*. Sesuai dengan karakteristik teoretisnya, bus I2C pada FIRESENTRY merupakan jenis komunikasi serial sinkron berbasis kabel (*wired*) yang menghemat penggunaan pin I/O secara signifikan karena hanya bergantung pada dua jalur bus utama, yaitu SDA (*Serial Data*) untuk jalur pengiriman bit data informasi, serta SCL (*Serial Clock*) yang dialiri sinyal detak konstan dari *Master* untuk menyelaraskan waktu pengiriman bit antara kedua perangkat agar tidak terjadi *overlapping*. Melalui sinkronisasi detak pada bus I2C inilah, ESP32 dapat mendorong pembaruan karakter dan status visual pada OLED secara cepat dalam hitungan milidetik. Informasi digital yang dilewatkan pada bus I2C ini mencakup tulisan status tingkat keamanan sirkuit lokal ("AMAN", "SIAGA", atau "BAHAYA") serta visualisasi angka numerik hasil pembacaan parameter lingkungan agar dapat langsung dipantau oleh operator atau penghuni di sekitar panel alat sebagai panduan mitigasi instan.

Protokol komunikasi kabel lokal yang kedua adalah OneWire, sebuah protokol komunikasi serial khusus yang diterapkan secara eksklusif untuk menghubungkan ESP32 dengan sensor suhu digital DS18B20 *waterproof*. Merujuk pada prinsip dasarnya, karakteristik utama dari bus OneWire ini adalah kemampuan transmisi data *master-to-slave* yang sangat efisien karena hanya mengeksploitasi satu jalur kabel data tunggal (*single data line*) untuk melewatkan bit informasi, di luar jalur catu daya fisik (*VCC* dan *GND*). Keunggulan teknis utama dari implementasi OneWire pada FIRESENTRY terletak pada arsitektur sensor DS18B20 yang sudah memiliki konverter internal, sehingga paket data temperatur yang dikirimkan melalui kabel tunggal tersebut sudah berbentuk representasi data digital murni berspesifikasi bit tertentu (*digital data stream*), bukan lagi berupa tegangan listrik analog yang fluktuatif. Dampaknya, unit pemroses ESP32 tidak perlu membuang

siklus kerja CPU untuk mengeksekusi konversi *Analog-to-Digital Converter* (ADC) eksternal pada pin mikrokontroler, sehingga meningkatkan efisiensi daya komputasi keseluruhan.

Selain memangkas beban kerja prosesor, transmisi data digital murni melalui protokol OneWire ini memiliki tingkat ketahanan yang sangat tinggi (*high robustness*) terhadap gangguan interferensi gelombang elektromagnetik (EMI) maupun derau (*noise*) frekuensi tinggi yang umumnya timbul di sekitar jaringan kabel daya tinggi atau panel distribusi listrik rumah tangga. Namun, karena bus OneWire mengandalkan satu jalur kabel yang dikonsumsi bersama, terdapat risiko terjadinya pelemahan kualitas sinyal digital (*signal degradation*) akibat kapasitas kabel. Untuk mengatasi keterbatasan fisik transmisi ini, sirkuit perangkat keras FIRESENTRY secara khusus mengintegrasikan sebuah resistor pembantu eksternal berupa resistor *pull-up* sebesar 4,7 kΩ yang menghubungkan jalur kabel data OneWire secara langsung ke sumber tegangan positif 5V. Pemasangan resistor *pull-up* ini berfungsi untuk menarik tegangan bus kembali ke level *High* secara cepat ketika kondisi *idle*, menjaga integritas bentuk gelombang kotak digital, dan meminimalkan risiko kehilangan bit data (*packet loss*) selama proses transfer data suhu berlangsung dari sensor ke mikrokontroler.

2.6 MESSAGE QUEUING TELEMETRY TRANSPORT (MQTT)

Sistem FIRESENTRY dirancang sebagai sistem Internet of Things (IoT) yang memanfaatkan protokol Message Queuing Telemetry Transport (MQTT) sebagai mekanisme komunikasi data antara perangkat ESP32 dan platform pemantauan berbasis cloud. MQTT dipilih karena merupakan protokol komunikasi ringan (lightweight protocol) yang dirancang khusus untuk perangkat dengan sumber daya terbatas dan jaringan yang memiliki bandwidth rendah. Pada implementasinya, ESP32 berperan sebagai publisher yang mengirimkan data hasil pembacaan sensor gas MQ-2, sensor suhu DS18B20, dan flame sensor ke MQTT Broker melalui jaringan WiFi. Seluruh data sensor yang dikumpulkan kemudian dikirimkan dalam format JSON sehingga dapat diproses dan ditampilkan secara real-time pada dashboard monitoring.

Dalam arsitektur MQTT, broker berfungsi sebagai perantara komunikasi antara perangkat pengirim data (publisher) dan perangkat penerima data (subscriber). Pada sistem FIRESENTRY, digunakan MQTT Broker berbasis cloud yang memungkinkan data sensor dapat diakses dari berbagai lokasi tanpa harus terhubung langsung ke jaringan lokal perangkat. Ketika ESP32 berhasil terhubung ke broker, setiap data yang dipublikasikan ke topik tertentu akan diteruskan oleh broker kepada seluruh subscriber yang berlangganan pada topik tersebut. Mekanisme ini memungkinkan proses pertukaran data berlangsung secara efisien karena publisher dan subscriber tidak perlu mengetahui alamat atau identitas satu sama lain. Dengan demikian, sistem menjadi lebih fleksibel, mudah dikembangkan, dan mampu mendukung pemantauan jarak jauh secara real-time.

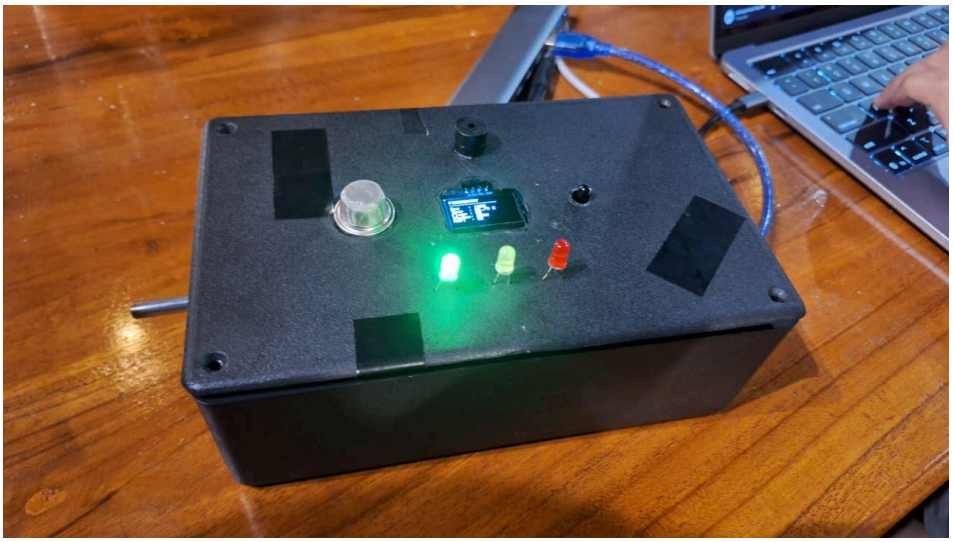
Penggunaan MQTT Broker pada FIRESENTRY juga meningkatkan keandalan sistem dalam proses monitoring kebakaran. Data hasil pembacaan sensor dikirim secara berkala sesuai kondisi sistem, yaitu dengan interval yang lebih jarang pada kondisi aman dan interval

yang lebih cepat ketika terdeteksi kondisi waspada atau bahaya. Pendekatan ini membantu mengoptimalkan penggunaan bandwidth jaringan sekaligus memastikan informasi penting dapat diterima oleh dashboard monitoring sesegera mungkin. Selain itu, fitur keamanan yang disediakan broker, seperti autentikasi username dan password serta penggunaan koneksi TLS/SSL, memungkinkan proses transmisi data berlangsung secara aman. Melalui penerapan MQTT Broker, FIRESENTRY mampu menyediakan sistem pemantauan kebakaran yang efisien, responsif, dan dapat diakses secara real-time melalui platform cloud.

BAB III

IMPLEMENTASI SISTEM

3.1 IMPLEMENTASI HARDWARE



Gambar 7. Implementasi Hardware

3.2 IMPLEMENTASI SOFTWARE

Kode Program ESP 31

```
#include <Wire.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>

#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

#include <OneWire.h>
#include <DallasTemperature.h>

// ====================
// WIFI SETUP
// ====================
const char* WIFI_SSID = "modalta";
const char* WIFI_PASSWORD = "nopal1234";

// ====================
// MQTT SETUP
// ====================
const char* MQTT_HOST =
```

```
"ef8001243aab4e788208799a8704d371.s1.eu.hivemq.cloud";
const int MQTT_PORT = 8883;

const char* MQTT_USERNAME = "nopal";
const char* MQTT_PASSWORD = "Nopal123";

const char* MQTT_TOPIC_SENSOR = "/firesentry/data";

// =====================
// MQTT INTERVAL
// =====================
// AMAN publish tiap 30 detik
const unsigned long MQTT_NORMAL_INTERVAL = 30000;

// ALERT dan BAHAYA publish tiap 1 detik
const unsigned long MQTT_ALERT_INTERVAL = 1000;
const unsigned long MQTT_DANGER_INTERVAL = 1000;

// Reconnect WiFi setiap 10 detik jika gagal
const unsigned long WIFI_RECONNECT_INTERVAL = 10000;

// Reconnect MQTT setiap 5 detik jika gagal
const unsigned long MQTT_RECONNECT_INTERVAL = 5000;

// =====================
// PIN SETUP
// =====================
#define PIN_DS18B20 4
#define PIN_MQ2 35
#define PIN_FLAME 18
#define PIN_LED_R 12
#define PIN_LED_G 14
#define PIN_LED_Y 13
#define PIN_BUZZER 19

// Banyak flame sensor module aktif LOW
// LOW  = api terdeteksi
// HIGH = tidak ada api
#define FLAME_DETECTED LOW

// =====================
// THRESHOLD SENSOR
// =====================
const int mq2AlertThreshold = 3950;
const float tempAlertThreshold = 40.0;

const int mq2DangerThreshold = 4090;
const float tempDangerThreshold = 60.0;
```

```
// ====================
// DS18B20 SETUP
// ====================
OneWire oneWire(PIN_DS18B20);
DallasTemperature sensors(&oneWire);

// ====================
// OLED SETUP
// ====================
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
OLED_RESET);

// ====================
// MQTT CLIENT SETUP
// ====================
WiFiClientSecure secureClient;
PubSubClient mqttClient(secureClient);

unsigned long lastMqttPublish = 0;
unsigned long lastMqttReconnectAttempt = 0;
unsigned long lastWiFiReconnectAttempt = 0;

String previousStateText = "";

// ====================
// STATE MACHINE
// ====================
enum SystemState {
  STATE_NORMAL,
  STATE_ALERT,
  STATE_EMERGENCY
};

SystemState currentState = STATE_NORMAL;

// ====================
// BUZZER TIMING
// ====================
unsigned long lastBuzzerToggle = 0;
bool buzzerOn = false;

// ====================
// LED CONTROL
// ====================
void turnOffAllLed() {
```

```
  digitalWrite(PIN_LED_R, LOW);
  digitalWrite(PIN_LED_Y, LOW);
  digitalWrite(PIN_LED_G, LOW);
}

void setLedNormal() {
  turnOffAllLed();
  digitalWrite(PIN_LED_G, HIGH);
}

void setLedAlert() {
  turnOffAllLed();
  digitalWrite(PIN_LED_Y, HIGH);
}

void setLedEmergency() {
  turnOffAllLed();
  digitalWrite(PIN_LED_R, HIGH);
}

// =====================
// STATUS TEXT
// =====================
String getStateText(SystemState state) {
  switch (state) {
    case STATE_NORMAL:
      return "AMAN";
    case STATE_ALERT:
      return "ALERT";
    case STATE_EMERGENCY:
      return "BAHAYA";
    default:
      return "UNKNOWN";
  }
}

// =====================
// WIFI CONNECT
// =====================
void connectWiFi() {
  if (WiFi.status() == WL_CONNECTED) {
    return;
  }

  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

  WiFi.mode(WIFI_STA);
```

```
  // Jangan panggil WiFi.disconnect(true) terus-menerus di
loop.
  // Cukup begin ulang berdasarkan interval reconnect.
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
}

// =====================
// WIFI STATUS CHECK
// =====================
void handleWiFiConnection() {
  if (WiFi.status() == WL_CONNECTED) {
    return;
  }

  unsigned long now = millis();

  if (now - lastWiFiReconnectAttempt >=
WIFI_RECONNECT_INTERVAL) {
    lastWiFiReconnectAttempt = now;
    connectWiFi();
  }
}

// =====================
// MQTT CONNECT
// =====================
bool connectMQTT() {
  if (WiFi.status() != WL_CONNECTED) {
    return false;
  }

  if (mqttClient.connected()) {
    return true;
  }

  Serial.println("Connecting to MQTT broker...");

  String clientId = "firesentry-esp32-";
  clientId += String((uint32_t)ESP.getEfuseMac(), HEX);

  bool connected = mqttClient.connect(
    clientId.c_str(),
    MQTT_USERNAME,
    MQTT_PASSWORD);

  if (connected) {
    Serial.println("MQTT connected!");
  } else {
```

```
    Serial.print("MQTT connection failed, rc=");
    Serial.println(mqttClient.state());
  }

  return connected;
}

// =====================
// MQTT STATUS CHECK
// =====================
void handleMQTTConnection() {
  if (WiFi.status() != WL_CONNECTED) {
    return;
  }

  if (mqttClient.connected()) {
    mqttClient.loop();
    return;
  }

  unsigned long now = millis();

  if (now - lastMqttReconnectAttempt >=
MQTT_RECONNECT_INTERVAL) {
    lastMqttReconnectAttempt = now;
    connectMQTT();
  }
}

// =====================
// PUBLISH MQTT PAYLOAD
// =====================
void publishSensorData(
  int mq2Value,
  float tempC,
  bool flameDetected,
  bool tempError,
  bool gasAlert,
  bool gasDanger,
  bool tempAlert,
  bool tempDanger,
  String stateText) {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("MQTT publish skipped: WiFi not
connected");
    return;
  }

  if (!mqttClient.connected()) {
```

```
    Serial.println("MQTT publish skipped: MQTT not
connected");
    return;
  }

  String payload = "{";

  payload += "\"gas\":";
  payload += String(mq2Value);
  payload += ",";

  payload += "\"temperature\":";
  if (tempError) {
    payload += "null";
  } else {
    payload += String(tempC, 2);
  }
  payload += ",";

  payload += "\"flameDetected\":";
  payload += flameDetected ? "true" : "false";
  payload += ",";

  payload += "\"tempError\":";
  payload += tempError ? "true" : "false";
  payload += ",";

  payload += "\"gasAlert\":";
  payload += gasAlert ? "true" : "false";
  payload += ",";

  payload += "\"gasDanger\":";
  payload += gasDanger ? "true" : "false";
  payload += ",";

  payload += "\"tempAlert\":";
  payload += tempAlert ? "true" : "false";
  payload += ",";

  payload += "\"tempDanger\":";
  payload += tempDanger ? "true" : "false";
  payload += ",";

  payload += "\"state\":\"";
  payload += stateText;
  payload += "\"";

  payload += "}";
```

```
  bool success = mqttClient.publish(MQTT_TOPIC_SENSOR,
payload.c_str());

  if (success) {
    Serial.print("MQTT Published: ");
    Serial.println(payload);
  } else {
    Serial.println("MQTT publish failed");
  }
}

// =====================
// OLED UPDATE
// =====================
void updateOLED(
  int mq2Value,
  float tempC,
  bool flameDetected,
  bool tempError,
  String stateText) {
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0, 0);

  display.println("FIRESENTRY");
  display.println("----------------");

  display.print("Gas   : ");
  display.println(mq2Value);

  display.print("Temp  : ");
  if (tempError) {
    display.println("ERROR");
  } else {
    display.print(tempC, 2);
    display.println(" C");
  }

  display.print("Flame : ");
  display.println(flameDetected ? "YES" : "NO");

  display.print("Status: ");
  display.println(stateText);

  display.print("MQTT  : ");
  display.println(mqttClient.connected() ? "OK" : "OFF");

  display.display();
```

```
}

// =====================
// GET MQTT INTERVAL BY STATE
// =====================
unsigned long getMqttIntervalByState(String stateText) {
  if (stateText == "AMAN") {
    return MQTT_NORMAL_INTERVAL;
  }

  if (stateText == "ALERT") {
    return MQTT_ALERT_INTERVAL;
  }

  if (stateText == "BAHAYA") {
    return MQTT_DANGER_INTERVAL;
  }

  return MQTT_NORMAL_INTERVAL;
}

// =====================
// SETUP
// =====================
void setup() {
  Serial.begin(115200);
  delay(1000);

  Serial.println();
  Serial.println("FIRESENTRY Integrated System");
  Serial.println("Mode: MQTT Hybrid Interval");
  Serial.println("AMAN: 30 detik | ALERT/BAHAYA: 1 detik");

  // OLED I2C ESP32 default:
  // SDA = GPIO 21
  // SCL = GPIO 22
  Wire.begin(21, 22);

  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED tidak terdeteksi. Cek alamat I2C /
wiring SDA SCL.");
    while (true) {
      delay(100);
    }
  }

  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
```

```
  display.setCursor(0, 0);
  display.println("FIRESENTRY");
  display.println("Initializing...");
  display.display();

  // DS18B20
  sensors.begin();
  sensors.setResolution(10);

  int jumlahSensor = sensors.getDeviceCount();
  Serial.print("Jumlah DS18B20 terdeteksi: ");
  Serial.println(jumlahSensor);

  // Pin mode sensor
  pinMode(PIN_MQ2, INPUT);
  pinMode(PIN_FLAME, INPUT);

  // Pin mode output
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_LED_R, OUTPUT);
  pinMode(PIN_LED_Y, OUTPUT);
  pinMode(PIN_LED_G, OUTPUT);

  digitalWrite(PIN_BUZZER, LOW);
  turnOffAllLed();

  // TLS MQTT
  // Untuk testing, setInsecure lebih mudah.
  // Untuk produksi, sebaiknya gunakan CA certificate.
  secureClient.setInsecure();

  mqttClient.setServer(MQTT_HOST, MQTT_PORT);
  mqttClient.setBufferSize(512);

  // Connect WiFi awal
  connectWiFi();

  delay(1000);
}

// =====================
// LOOP
// =====================
void loop() {
  // =====================
  // 1. Jaga koneksi WiFi & MQTT
  // =====================
  handleWiFiConnection();
```

```
  if (WiFi.status() == WL_CONNECTED) {
    handleMQTTConnection();
  }

  // ====================
  // 2. READ SENSOR
  // ====================
  sensors.requestTemperatures();
  float tempC = sensors.getTempCByIndex(0);

  int mq2Value = analogRead(PIN_MQ2);
  int flameState = digitalRead(PIN_FLAME);

  bool flameDetected = (flameState == FLAME_DETECTED);
  bool tempError = (tempC == DEVICE_DISCONNECTED_C);

  // Kalau DS18B20 error, suhu tidak dipakai untuk
pengambilan keputusan
  bool tempAlert = (!tempError && tempC >
tempAlertThreshold);
  bool tempDanger = (!tempError && tempC >
tempDangerThreshold);

  bool gasAlert = (mq2Value > mq2AlertThreshold);
  bool gasDanger = (mq2Value > mq2DangerThreshold);

  // ====================
  // 3. STATE DECISION
  // BAHAYA dicek dulu karena paling prioritas
  // ====================
  if (flameDetected || gasDanger || tempDanger) {
    currentState = STATE_EMERGENCY;
  } else if (gasAlert || tempAlert) {
    currentState = STATE_ALERT;
  } else {
    currentState = STATE_NORMAL;
  }

  String stateText = getStateText(currentState);

  // ====================
  // 4. OUTPUT CONTROL
  // ====================
  unsigned long now = millis();

  switch (currentState) {
    case STATE_NORMAL:
      setLedNormal();
      digitalWrite(PIN_BUZZER, LOW);
```

```
      buzzerOn = false;
      break;

    case STATE_ALERT:
      setLedAlert();

      // Buzzer lambat: beep setiap 500 ms
      if (now - lastBuzzerToggle >= 500) {
        buzzerOn = !buzzerOn;
        digitalWrite(PIN_BUZZER, buzzerOn ? HIGH : LOW);
        lastBuzzerToggle = now;
      }
      break;

    case STATE_EMERGENCY:
      setLedEmergency();

      // Buzzer cepat: beep setiap 150 ms
      if (now - lastBuzzerToggle >= 150) {
        buzzerOn = !buzzerOn;
        digitalWrite(PIN_BUZZER, buzzerOn ? HIGH : LOW);
        lastBuzzerToggle = now;
      }
      break;
  }

  // =====================
  // 5. OLED DISPLAY
  // =====================
  updateOLED(
    mq2Value,
    tempC,
    flameDetected,
    tempError,
    stateText);

  // =====================
  // 6. SERIAL DEBUG
  // =====================
  Serial.print("WiFi: ");
  Serial.print(WiFi.status() == WL_CONNECTED ? "OK" : "OFF");

  Serial.print(" | MQTT: ");
  Serial.print(mqttClient.connected() ? "OK" : "OFF");

  Serial.print(" | Gas: ");
  Serial.print(mq2Value);

  Serial.print(" | Temp: ");
```

```
  if (tempError) {
    Serial.print("ERROR");
  } else {
    Serial.print(tempC, 2);
    Serial.print(" C");
  }

  Serial.print(" | Flame: ");
  Serial.print(flameDetected ? "YES" : "NO");

  Serial.print(" | gasAlert: ");
  Serial.print(gasAlert ? "true" : "false");

  Serial.print(" | gasDanger: ");
  Serial.print(gasDanger ? "true" : "false");

  Serial.print(" | tempAlert: ");
  Serial.print(tempAlert ? "true" : "false");

  Serial.print(" | tempDanger: ");
  Serial.print(tempDanger ? "true" : "false");

  Serial.print(" | Status: ");
  Serial.println(stateText);

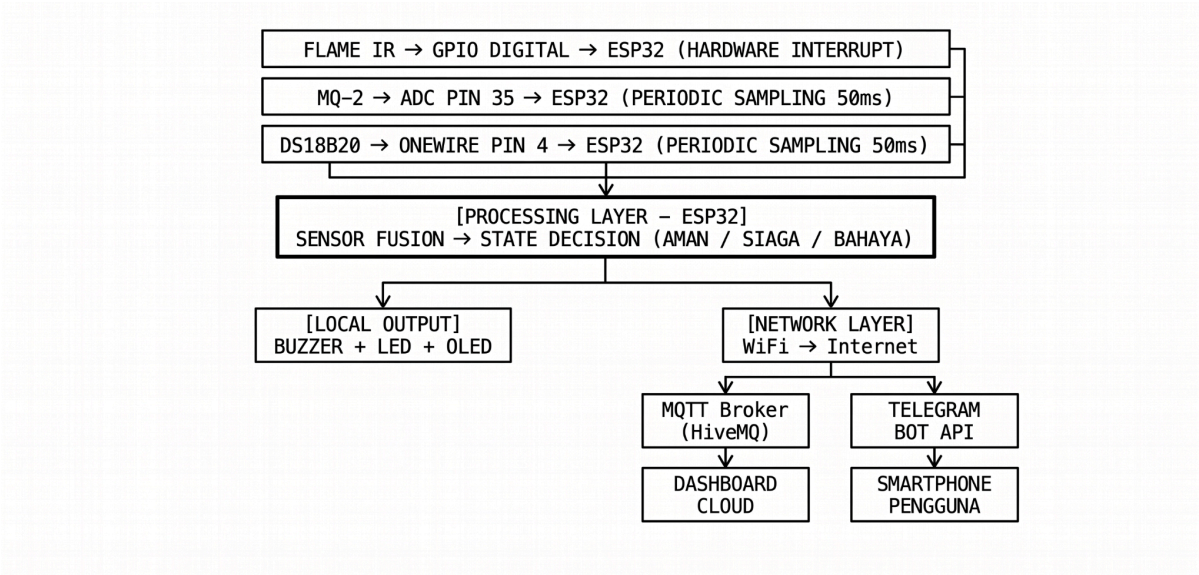
  // ====================
  // 7. MQTT PUBLISH HYBRID
  // AMAN    -> tiap 30 detik
  // ALERT   -> tiap 1 detik
  // BAHAYA  -> tiap 1 detik
  // Jika state berubah -> publish langsung
  // ====================
  bool stateChanged = (stateText != previousStateText);
  unsigned long mqttInterval =
getMqttIntervalByState(stateText);

  if (mqttClient.connected()) {
    if (stateChanged || (millis() - lastMqttPublish >=
mqttInterval)) {
      publishSensorData(
        mq2Value,
        tempC,
        flameDetected,
        tempError,
        gasAlert,
        gasDanger,
        tempAlert,
        tempDanger,
        stateText);
```

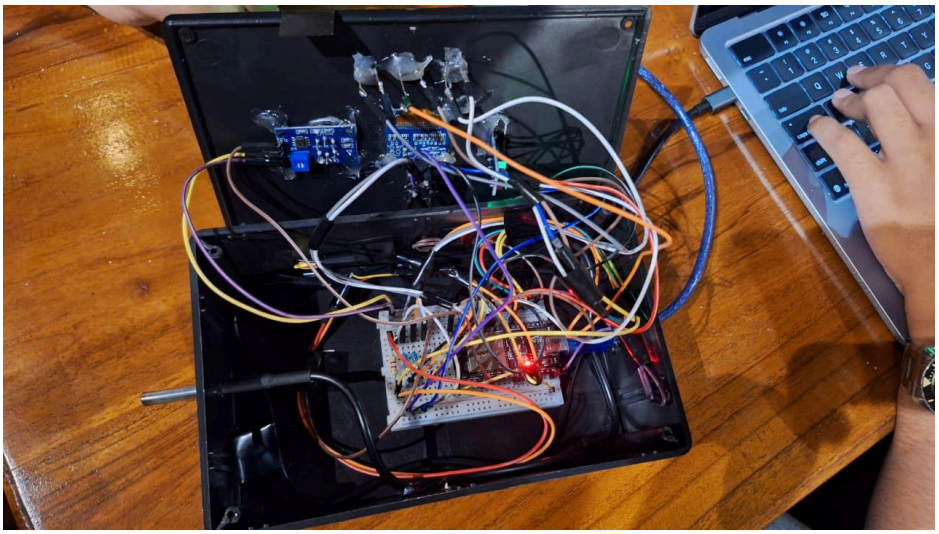
```
      lastMqttPublish = millis();
      previousStateText = stateText;
    }
  }

  delay(50);
}
```

3.3 INTEGRASI SENSOR



Gambar 8. Topologi Urutan Pengiriman Data Sensor



Gambar 9. Integrasi Sensor

Sistem FIRESENTRY menggunakan tiga sensor utama, yaitu Flame IR Sensor, MQ-2 Gas Sensor, dan DS18B20 Temperature Sensor sebagai sumber data untuk mendeteksi potensi kebakaran. Flame IR Sensor berfungsi untuk mendeteksi keberadaan api melalui radiasi inframerah yang dipancarkan oleh nyala api. Sensor MQ-2 digunakan untuk mengukur konsentrasi gas dan asap yang dapat menjadi indikasi awal terjadinya kebakaran, sedangkan sensor DS18B20 digunakan untuk memantau suhu lingkungan secara real-time. Ketiga sensor tersebut terhubung langsung ke mikrokontroler ESP32 yang berperan sebagai pusat kendali dan pengolahan data dalam sistem.

Proses kerja sistem dimulai dengan pembacaan data dari masing-masing sensor secara berkala menggunakan metode polling. ESP32 melakukan pembacaan nilai analog dari sensor MQ-2 melalui pin ADC, membaca data suhu dari sensor DS18B20 melalui protokol komunikasi OneWire, serta membaca status deteksi api dari Flame IR Sensor melalui pin digital. Data yang diperoleh dari ketiga sensor kemudian dikumpulkan dan diproses secara terus-menerus untuk memperoleh kondisi lingkungan terkini. Proses pembacaan dilakukan dalam interval yang relatif singkat sehingga perubahan kondisi lingkungan dapat terdeteksi dengan cepat.

Data hasil pembacaan sensor selanjutnya masuk ke lapisan pemrosesan pada ESP32. Pada tahap ini dilakukan proses sensor fusion, yaitu penggabungan informasi dari beberapa sensor untuk menghasilkan keputusan yang lebih akurat. Sistem membandingkan nilai suhu, konsentrasi gas, dan status deteksi api dengan nilai ambang batas yang telah ditentukan sebelumnya. Berdasarkan hasil analisis tersebut, sistem akan menentukan status operasional yang terdiri dari tiga kondisi, yaitu AMAN, SIAGA (ALERT), dan BAHAYA. Kondisi aman menunjukkan bahwa seluruh parameter masih berada pada rentang normal. Kondisi siaga menunjukkan adanya peningkatan salah satu parameter yang mendekati batas bahaya, sedangkan kondisi bahaya menunjukkan terdeteksinya api atau parameter sensor telah melewati batas kritis yang mengindikasikan kemungkinan terjadinya kebakaran.

Setelah status sistem ditentukan, ESP32 akan mengaktifkan perangkat keluaran lokal sebagai media peringatan langsung kepada pengguna. Pada kondisi aman, LED hijau akan menyala sebagai indikator bahwa lingkungan berada dalam keadaan normal. Ketika sistem memasuki kondisi siaga, LED kuning akan aktif sebagai tanda bahwa terdapat parameter yang perlu diwaspadai. Apabila sistem mendeteksi kondisi bahaya, LED merah akan menyala dan buzzer akan diaktifkan untuk memberikan peringatan suara kepada pengguna. Selain itu, informasi suhu dan status sistem juga ditampilkan pada layar OLED 0,96 inci sehingga pengguna dapat memantau kondisi lingkungan secara langsung pada perangkat tanpa memerlukan akses ke sistem monitoring eksternal.

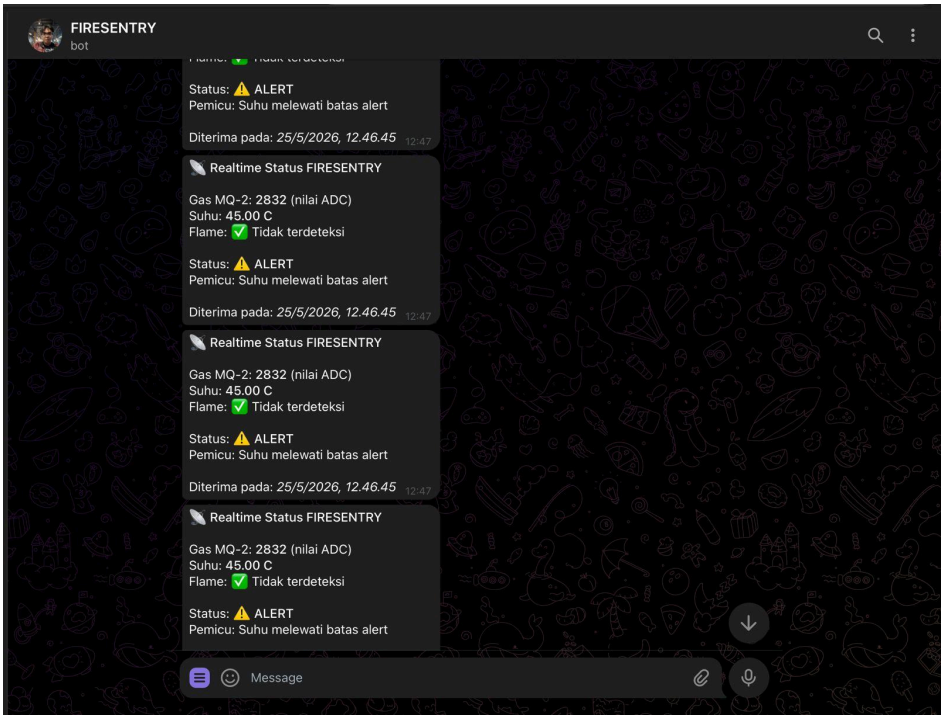
Selain memberikan peringatan lokal, ESP32 juga mengirimkan data hasil pemantauan melalui jaringan WiFi menggunakan protokol MQTT. Data sensor dan status sistem dipublikasikan ke MQTT Broker HiveMQ dalam format JSON. MQTT Broker berfungsi sebagai perantara komunikasi yang menerima data dari ESP32 sebagai publisher dan meneruskannya kepada perangkat lain yang berlangganan pada topik yang sama sebagai

subscriber. Mekanisme ini memungkinkan proses pertukaran data berlangsung secara efisien tanpa memerlukan koneksi langsung antara perangkat pengirim dan penerima data.

Data yang telah diterima oleh MQTT Broker kemudian diteruskan ke dashboard monitoring berbasis cloud sehingga kondisi sistem dapat dipantau secara real-time dari lokasi yang berbeda. Dashboard menampilkan informasi penting seperti nilai suhu, konsentrasi gas, status deteksi api, serta kondisi sistem secara keseluruhan. Selain itu, sistem juga terintegrasi dengan Telegram Bot API yang digunakan untuk mengirimkan notifikasi otomatis ke smartphone pengguna ketika sistem mendeteksi kondisi siaga atau bahaya. Melalui mekanisme ini, pengguna dapat memperoleh informasi dan peringatan secara cepat meskipun tidak berada di dekat perangkat.

Integrasi antara sensor, ESP32, perangkat keluaran lokal, MQTT Broker, dashboard cloud, dan Telegram Bot API menghasilkan sebuah sistem deteksi kebakaran berbasis Internet of Things (IoT) yang mampu melakukan pemantauan lingkungan secara berkelanjutan. Sistem tidak hanya memberikan peringatan secara lokal melalui LED, buzzer, dan OLED, tetapi juga memungkinkan pemantauan dan notifikasi jarak jauh melalui internet. Dengan arsitektur tersebut, FIRESENTRY mampu meningkatkan kecepatan respons terhadap potensi kebakaran serta mendukung upaya mitigasi risiko melalui penyampaian informasi secara real-time kepada pengguna.

3.4 IMPLEMENTASI MONITORING DAN NOTIFIKASI TELEGRAM BOT



Gambar 10. Implementasi Monitoring dan Notifikasi Telegram Bot

Telegram Bot pada sistem FIRESENTRY berfungsi sebagai media notifikasi jarak jauh yang mengirimkan informasi kondisi lingkungan secara real-time kepada pengguna. Notifikasi yang ditampilkan memuat beberapa parameter penting hasil pembacaan sensor sehingga pengguna dapat mengetahui kondisi sistem tanpa harus melihat perangkat secara langsung.

Pada bagian atas notifikasi ditampilkan judul "Realtime Status FIRESENTRY" yang menunjukkan bahwa pesan yang diterima merupakan hasil pemantauan kondisi terkini dari sistem. Selanjutnya terdapat informasi Gas MQ-2 yang menunjukkan nilai pembacaan sensor gas/asap dalam satuan ADC (Analog-to-Digital Converter). Nilai ini digunakan untuk mengetahui tingkat konsentrasi gas atau asap yang terdeteksi oleh sensor MQ-2. Semakin tinggi nilai ADC yang diperoleh, semakin besar kemungkinan adanya peningkatan konsentrasi gas atau asap di lingkungan yang dipantau.

Informasi berikutnya adalah Suhu, yang menampilkan hasil pengukuran sensor DS18B20 dalam satuan derajat Celsius (°C). Data suhu digunakan untuk memantau kenaikan temperatur lingkungan yang dapat menjadi indikasi awal terjadinya kebakaran. Pada notifikasi yang ditampilkan, suhu tercatat sebesar 45,00°C, yang menunjukkan bahwa suhu telah melewati batas normal dan memasuki kondisi waspada sesuai konfigurasi sistem.

Selanjutnya terdapat informasi Flame, yang menunjukkan status deteksi api oleh Flame IR Sensor. Jika sensor mendeteksi adanya api, maka status akan berubah menjadi "Terdeteksi", sedangkan pada notifikasi yang ditampilkan statusnya adalah "Tidak terdeteksi", yang berarti sistem tidak menemukan radiasi inframerah yang mengindikasikan keberadaan nyala api pada saat pengukuran dilakukan.

Bagian terpenting dari notifikasi adalah informasi Status Sistem, yang menunjukkan hasil analisis keseluruhan berdasarkan data dari seluruh sensor. Pada contoh notifikasi, status yang ditampilkan adalah ALERT, yang berarti sistem mendeteksi kondisi yang perlu diwaspadai tetapi belum mencapai tingkat bahaya. Untuk membantu pengguna memahami penyebab perubahan status tersebut, sistem juga menampilkan informasi Pemicu, yaitu parameter yang menyebabkan sistem masuk ke kondisi tertentu. Pada notifikasi ini, pemicunya adalah "Suhu melewati batas alert", yang menunjukkan bahwa nilai suhu telah melampaui ambang batas peringatan yang telah ditentukan.

Terakhir, notifikasi menampilkan informasi "Diterima pada" yang berisi tanggal dan waktu pengiriman pesan. Informasi ini berfungsi sebagai penanda waktu (timestamp) sehingga pengguna dapat mengetahui kapan kondisi tersebut terdeteksi oleh sistem. Dengan adanya data sensor, status sistem, penyebab perubahan kondisi, serta waktu kejadian dalam satu notifikasi, Telegram Bot FIRESENTRY mampu memberikan informasi yang lengkap dan mudah dipahami untuk mendukung proses pemantauan dan respons dini terhadap potensi kebakaran.

BAB IV

ANALISIS DAN PENGUJIAN

4.1 RELIABILITY

Reliability pada sistem FIRESENTRY diwujudkan melalui penggunaan ESP32 sebagai mikrokontroler utama yang mampu mengelola proses pembacaan sensor, pengolahan data, serta pengiriman data secara terstruktur melalui komunikasi MQTT ke broker. Dengan mekanisme ini, data dari sistem dapat dikirim secara real-time dan terorganisir, sehingga proses monitoring kondisi lingkungan dapat dilakukan secara stabil dan berkelanjutan tanpa ketergantungan pada proses manual.

Selain dari sisi komunikasi, tingkat keandalan sistem juga ditingkatkan melalui penggunaan beberapa sensor yang bekerja secara bersamaan, yaitu DS18B20, MQ-2, dan Flame IR Sensor. Ketiga sensor ini digunakan dalam pendekatan sensor fusion untuk meningkatkan akurasi deteksi potensi kebakaran. Dengan adanya beberapa sumber data, sistem tidak hanya bergantung pada satu sensor saja, sehingga kemungkinan terjadinya kesalahan pembacaan dapat diminimalkan dan hasil deteksi menjadi lebih akurat serta lebih dapat dipercaya.

Dalam implementasinya, FIRESENTRY juga menerapkan konsep fault prevention untuk mengurangi potensi kesalahan sejak tahap perancangan sistem. Sensor DS18B20 menggunakan komunikasi digital OneWire yang lebih tahan terhadap gangguan elektromagnetik, sehingga pembacaan suhu menjadi lebih stabil. Selain itu, penggunaan resistor pull-up pada jalur data membantu menjaga kestabilan sinyal komunikasi antar komponen. Sistem juga menerapkan threshold dan filtering pada hasil pembacaan sensor untuk mengurangi noise serta meminimalkan terjadinya false alarm.

Selanjutnya, sistem didukung dengan proses pengujian secara berkala untuk memastikan seluruh sensor, modul komunikasi, dan aktuator bekerja sesuai dengan spesifikasi yang telah ditentukan. Pengujian ini menjadi bagian dari pendekatan fault removal, yaitu proses identifikasi dan perbaikan kesalahan yang muncul selama tahap pengembangan maupun implementasi sistem. Dengan adanya pengujian rutin, potensi gangguan dapat dideteksi lebih awal sebelum memengaruhi kinerja sistem secara keseluruhan.

Dari sisi keselamatan (safety), FIRESENTRY dirancang untuk memberikan respons cepat ketika kondisi berbahaya terdeteksi. Sistem akan segera mengaktifkan berbagai indikator peringatan seperti buzzer, LED, tampilan pada OLED display, serta mengirimkan notifikasi melalui Telegram yang terintegrasi dengan MQTT broker. Mekanisme ini memastikan pengguna dapat segera menerima informasi darurat secara cepat dan akurat sehingga dapat melakukan tindakan pencegahan.

Selain itu, sistem juga memiliki maintainability yang baik karena setiap sensor dan komponen dirancang secara modular. Dengan pendekatan ini, proses pemeriksaan, perawatan, maupun penggantian komponen dapat dilakukan dengan lebih mudah tanpa harus membongkar keseluruhan sistem. Hal ini membuat sistem lebih fleksibel, mudah dikembangkan, serta lebih efisien dalam jangka panjang.

4.2 VALIDASI DAN EVALUASI

Pada proyek FIRESENTRY, proses validation dilakukan untuk memastikan bahwa sistem deteksi dan mitigasi kebakaran dapat bekerja sesuai tujuan, memenuhi kebutuhan sistem, serta mampu memberikan respons yang tepat ketika terjadi kondisi bahaya. Validation dilakukan dengan menguji seluruh fungsi utama sistem, seperti pembacaan sensor suhu DS18B20, sensor gas MQ-2, Flame IR Sensor, pengaktifan buzzer dan LED, tampilan OLED, serta pengiriman notifikasi Telegram. Pengujian dilakukan dengan memberikan kondisi tertentu, seperti peningkatan suhu, keberadaan asap, dan nyala api untuk melihat apakah sistem mampu mendeteksi kondisi tersebut dengan benar. Proses ini penting karena FIRESENTRY merupakan sistem yang berkaitan dengan keselamatan (*safety-related system*), sehingga kesalahan deteksi dapat berdampak pada keterlambatan respon terhadap potensi kebakaran.

Metode validasi yang diterapkan pada FIRESENTRY meliputi *prototype and testing*. Sistem dibuat dalam bentuk prototipe menggunakan ESP32 dan seluruh sensor yang kemudian diuji secara langsung pada kondisi nyata untuk mengetahui apakah setiap komponen bekerja sesuai rancangan. Pengujian dilakukan dengan memberikan beberapa *test pattern*, seperti menyalakan api di dekat Flame IR Sensor, memberikan asap pada MQ-2, serta meningkatkan temperatur pada DS18B20. Hasil keluaran sistem, seperti aktivasi buzzer, perubahan status pada OLED, serta pengiriman notifikasi Telegram, kemudian dibandingkan dengan respons yang diharapkan.

Selain validation, proyek FIRESENTRY juga menerapkan proses evaluation untuk mengukur performa sistem secara kuantitatif. Evaluasi dilakukan terhadap beberapa aspek, seperti kecepatan respons sistem, akurasi deteksi sensor, konsumsi daya ESP32, stabilitas komunikasi IoT melalui MQTT broker, serta kemampuan sistem dalam meminimalkan false alarm melalui pendekatan *sensor fusion*. Penggunaan MQTT broker memungkinkan komunikasi data antara perangkat dan server berjalan secara real-time dengan latensi yang rendah, sehingga notifikasi kondisi darurat dapat dikirim lebih cepat dan stabil.

FIRESENTRY juga dilakukan analisis risiko sederhana untuk mengidentifikasi potensi kegagalan sistem, seperti keterlambatan pengiriman data melalui MQTT, kesalahan pembacaan sensor, atau gangguan koneksi jaringan. Analisis ini bertujuan untuk meningkatkan keandalan sistem secara keseluruhan agar tetap dapat memberikan peringatan dini kebakaran secara konsisten dalam berbagai kondisi lingkungan.

4.3 OPTIMASI SISTEM

Sistem dirancang sebagai solusi pemantauan dan deteksi dini kebakaran yang mampu mengamati kondisi lingkungan secara real-time melalui beberapa parameter penting, yaitu konsentrasi gas, suhu lingkungan, dan keberadaan api. Untuk memperoleh informasi tersebut, digunakan sensor MQ-2 sebagai pendeteksi gas dan asap, sensor DS18B20 sebagai pengukur suhu, serta flame sensor untuk mendeteksi nyala api secara langsung. Selain melakukan pembacaan sensor, mikrokontroler ESP32 juga harus menjalankan berbagai tugas lainnya, seperti mengolah data hasil pembacaan sensor, menentukan status sistem berdasarkan kondisi yang terdeteksi, mengendalikan LED indikator dan buzzer sebagai perangkat peringatan, memperbarui informasi pada layar OLED, menjaga konektivitas jaringan WiFi, serta mengirimkan data pemantauan ke server melalui protokol MQTT. Banyaknya fungsi yang harus dijalankan secara bersamaan menyebabkan diperlukan suatu metode pengambilan data yang sederhana, stabil, dan mudah diintegrasikan dengan seluruh komponen sistem. Oleh karena itu, metode polling dipilih sebagai mekanisme utama dalam proses pemantauan sensor karena mampu mendukung seluruh kebutuhan sistem dalam satu alur program yang terstruktur dan mudah dikendalikan.

Metode polling bekerja dengan cara mikrokontroler secara aktif memeriksa atau membaca status setiap sensor secara berkala di dalam fungsi utama program (loop()). Pada implementasi FireSentry, ESP32 melakukan pembacaan sensor gas MQ-2, sensor suhu DS18B20, dan flame sensor secara terus-menerus dengan interval yang relatif singkat, yaitu sekitar 50 milidetik. Hasil pembacaan tersebut kemudian digunakan untuk menentukan kondisi sistem berdasarkan logika state machine yang terdiri dari tiga keadaan, yaitu AMAN, ALERT, dan BAHAYA. Pendekatan ini memungkinkan seluruh data sensor diperbarui secara berkala sehingga sistem dapat memberikan gambaran kondisi lingkungan yang selalu terkini. Selain itu, penggunaan polling juga memudahkan sinkronisasi antara proses pembacaan sensor dengan proses lainnya, seperti pengiriman data MQTT, pembaruan tampilan OLED, serta pengendalian alarm dan indikator visual. Dengan demikian, seluruh komponen sistem dapat bekerja secara terkoordinasi tanpa memerlukan mekanisme komunikasi yang kompleks antar proses.

Pemilihan metode polling juga didasarkan pada karakteristik sensor yang digunakan dalam proyek ini. Sensor MQ-2 menghasilkan keluaran analog yang menunjukkan tingkat konsentrasi gas di lingkungan sekitar. Nilai tersebut perlu dibaca secara terus-menerus untuk mengetahui apakah terjadi peningkatan kadar gas yang mengindikasikan adanya potensi kebakaran. Sementara itu, sensor DS18B20 memerlukan proses komunikasi digital menggunakan protokol OneWire untuk memperoleh data suhu, sehingga pembacaannya dilakukan melalui permintaan data secara berkala. Kedua sensor tersebut tidak menghasilkan sinyal kejadian (event) yang dapat langsung memicu interrupt ketika terjadi perubahan nilai. Dengan kata lain, meskipun sistem menggunakan interrupt, sensor tetap harus dibaca secara periodik untuk memperoleh informasi yang dibutuhkan. Oleh karena itu, polling menjadi pilihan yang lebih logis dan efisien karena sesuai dengan cara kerja sensor yang digunakan. Selain itu, perubahan suhu maupun konsentrasi gas pada lingkungan umumnya berlangsung

secara bertahap dalam hitungan detik, sehingga tidak memerlukan respons dalam skala mikrodetik sebagaimana pada aplikasi kontrol industri berkecepatan tinggi.

Apabila dibandingkan dengan metode interrupt, polling memiliki beberapa keunggulan yang membuatnya lebih sesuai untuk diterapkan pada FireSentry. Metode interrupt bekerja dengan cara menghentikan sementara alur program utama ketika terdapat sinyal pemicu dari perangkat keras, kemudian menjalankan Interrupt Service Routine (ISR) untuk menangani kejadian tersebut. Pendekatan ini sangat efektif pada aplikasi yang membutuhkan respons sangat cepat terhadap perubahan kondisi tertentu, seperti pembacaan encoder, pendeteksian tombol, atau sistem kendali waktu nyata (real-time control). Namun, penggunaan interrupt pada FireSentry tidak memberikan manfaat yang signifikan karena sebagian besar sensor yang digunakan tidak menghasilkan sinyal interrupt secara langsung dan tetap memerlukan proses pembacaan berkala. Selain itu, ISR memiliki berbagai keterbatasan, seperti tidak dianjurkan menjalankan proses yang memerlukan waktu lama, komunikasi jaringan, atau akses ke perangkat tertentu karena dapat mengganggu stabilitas sistem. Pada proyek ini, ESP32 harus menangani komunikasi WiFi dan MQTT yang relatif kompleks sehingga penggunaan interrupt justru berpotensi menambah kompleksitas pengembangan perangkat lunak tanpa memberikan peningkatan performa yang berarti.

Dari sisi pengembangan dan pemeliharaan sistem, metode polling juga menawarkan kemudahan yang lebih besar dibandingkan interrupt. Alur program yang bersifat berurutan membuat proses debugging, pengujian, dan pengembangan fitur menjadi lebih sederhana karena seluruh proses dapat ditelusuri secara langsung pada fungsi utama program. Risiko terjadinya konflik akses data (race condition), sinkronisasi variabel, maupun masalah konkurensi juga dapat diminimalkan karena tidak terdapat proses ISR yang berjalan secara asinkron. Hal ini sangat penting dalam proyek FireSentry yang mengintegrasikan berbagai perangkat keras dan layanan jaringan dalam satu sistem terpadu. Dengan struktur program yang lebih sederhana, pengembangan sistem menjadi lebih mudah dilakukan serta meminimalkan kemungkinan terjadinya kesalahan yang dapat mempengaruhi keandalan sistem deteksi kebakaran.

Metode polling dipilih sebagai solusi yang paling sesuai karena karakteristik sensor yang memerlukan pembacaan berkala, kebutuhan integrasi dengan berbagai fungsi sistem, kemudahan implementasi, serta kemampuan memberikan respons yang cukup cepat terhadap perubahan kondisi lingkungan. Kecepatan pembacaan sensor yang relatif tinggi memungkinkan setiap perubahan kondisi lingkungan terdeteksi dalam waktu singkat, sehingga sistem mampu melakukan identifikasi dini terhadap potensi kebakaran melalui parameter suhu, konsentrasi gas, dan keberadaan api secara real-time. Oleh karena itu, penggunaan polling tidak hanya memberikan kemudahan dalam implementasi, tetapi juga mampu memenuhi kebutuhan performa dan keandalan sistem secara keseluruhan.

4.4 TESTING

4.4.1 TABEL PENGUJIAN JARAK DETEKSI FLAME IR SENSOR

No	Jarak Sumber Api terhadap Sensor	Kondisi Pengujian	Hasil Pembacaan Flame Sensor	Respons Sistem	Waktu Respons	Keterangan
1.	5 cm	Api kecil	YES	LED merah ON, buzzer aktif	0,40 detik	Deteksi sangat cepat dan stabil
2.	10 cm	Api kecil	YES	LED merah ON, buzzer aktif	0,53 detik	Deteksi sangat cepat dan stabil
3.	30 cm	Api kecil	YES	LED merah ON, buzzer aktif	2,03 detik	Jarak maksimum efektif
4.	50 cm	Api kecil	NO	LED hijau ON, buzzer mati	-	Api tidak terdeteksi

4.4.2 TABEL PENGUJIAN SENSOR MQ-2 BERDASARKAN JARAK ASAP/GAS

No	Jarak Sumber Asap/Gas terhadap Sensor	Lama Paparan	Nilai ADC MQ-2	Status Sistem	Respons Output	Keterangan
1.	5 cm	5 detik	4200	BAHAYA	LED merah ON, buzzer bunyi cepat (150ms)	Nilai ADC > 4090, konsentrasi gas sangat tinggi, alarm penuh aktif
2.	30 cm	10 detik	4020	ALERT	LED	Nilai ADC

					kuning ON, buzzer bunyi lambat (500ms)	> 3950 dan ≤ 4090, konsentrasi gas terdeteksi, peringatan aktif
3.	50 cm	10 detik	3700	AMAN	LED hijau ON, buzzer OFF	Nilai ADC ≤ 3950, konsentrasi gas rendah, sistem normal
4.	100 cm	10 detik	3200	AMAN	LED hijau ON, buzzer OFF	Nilai ADC jauh di bawah threshold, sensor hampir tidak mendeteksi gas

4.4.3 TABEL PENGUJIAN SENSOR SUHU DS18B20 BERDASARKAN PERUBAHAN TEMPERATUR

No	Kondisi Pengujian	Suhu Target	Hasil Pembacaan DS18B20	Status Sistem	Respons Output	Waktu Respons	Keterangan
1.	Kondisi normal ruangan	25–35°C	32,25°C	AMAN	LED hijau, buzzer OFF	-	Kondisi normal
2.	Suhu meningkat	40°C	42,75°C	ALERT	LED kuning, buzzer intermittent	5 detik	Melewati threshold alert
3.	Suhu tinggi	50°C	51,50°C	ALERT	LED kuning, buzzer intermittent	7 detik	Melewati threshold alert

4.	Kondisi bahaya	>60°C	63,00°C	BAHAYA	LED merah, buzzer alarm aktif	10,5 detik	Kondisi darurat

4.4 TABEL PENGUJIAN KOMBINASI SENSOR

No	Kondisi Uji	Flame IR	MQ-2	DS18B20	Status Sistem Seharusnya	Hasil Sistem	Sesuai /Tidak
1.	Normal	Tidak terdeteksi	Rendah	<40°C	AMAN	AMAN	Sesuai
2.	Asap terdeteksi	Tidak terdeteksi	Tinggi	Normal	ALERT	ALERT	Sesuai
3.	Api terdeteksi	LOW	Normal	Normal	BAHAYA	BAHAYA	Sesuai
4.	Kondisi kebakaran	LOW	Tinggi	>60°C	BAHAYA	BAHAYA	Sesuai

4.4.5 TABEL PENGUJIAN NOTIFIKASI IOT (MQTT/TELEGRAM)

No	Kondisi Pengujian	Input Sensor	Status Sistem	Waktu Pengiriman Notifikasi	Hasil	Keterangan
1.	Kondisi normal	Suhu normal, gas rendah, tidak ada api	AMAN	±30 detik	Berhasil	Data monitoring terkirim
2.	Kondisi siaga	Gas/suhu melewati threshold	ALERT	±1 detik	Berhasil	Notifikasi peringatan diterima
3.	Kondisi bahaya	Api terdeteksi	BAHAYA	<1 detik	Berhasil	Notifikasi darurat diterima

4.4.6 TABEL PENGUJIAN RELIABILITY SISTEM

No	Durasi Pengujian	Kondisi Sistem	Pengujian	Hasil Sistem	Keterangan
1.	30 menit	Monitoring normal	Sistem berjalan tanpa gangguan	Stabil	Sensor dan output bekerja normal
2.	1 jam	Perubahan kondisi sensor	Input sensor berubah-ubah	Stabil	Sistem mampu merespons perubahan
3.	2 jam	Komunikasi IoT aktif	Pengiriman data MQTT berjalan	Stabil	Tidak terjadi koneksi terputus
4.	3 jam	Operasi kontinu	Sistem aktif tanpa reset	Stabil	Tidak terjadi error/crash

4.4.7 PERBANDINGAN MQTT VS TELEGRAM BOT API

Aspek	MQTT (HiveMQ Cloud)	Telegram Bot API (HTTPS)
Latensi rata-rata	1088,7 ms	5208 ms
Latensi minimum	317 ms	—
Latensi maksimum	3125 ms	—
Success rate	95,5% (21/22 pesan)	100% (1 pengujian)
Protokol	MQTT over TLS port 8883	HTTPS REST API
Overhead bandwidth	Sangat rendah (~2 byte header)	Tinggi (~200–800 byte HTTP header)
Koneksi	Persistent keep-alive	Request-response per pesan
Mekanisme antrian	QoS 0/1/2 + retain	Tidak ada
Cocok untuk	Data log periodik & monitoring	Notifikasi darurat ke pengguna
Kondisi pengujian	Hotspot smartphone	Hotspot smartphone

BAB V

PENUTUP

5.1 KESIMPULAN

Sistem FIRESENTRY berhasil dirancang dan diimplementasikan sebagai solusi mitigasi kebakaran dini berbasis embedded system dan IoT menggunakan mikrokontroler ESP32 yang terintegrasi dengan sensor Flame IR, MQ-2, dan DS18B20. Penerapan metode sensor fusion terbukti mampu meningkatkan akurasi deteksi kondisi bahaya sekaligus meminimalkan terjadinya false alarm, karena keputusan sistem didasarkan pada kombinasi data dari ketiga sensor secara bersamaan. Mekanisme respons multi-level yang mencakup indikator LED, buzzer, tampilan OLED, serta notifikasi real-time melalui Telegram Bot dan protokol MQTT memungkinkan pengguna memperoleh informasi kondisi lingkungan secara cepat baik dari jarak dekat maupun jauh. Hasil pengujian menunjukkan bahwa sistem mampu beroperasi secara stabil selama lebih dari tiga jam tanpa gangguan, dengan waktu respons deteksi api sesingkat 0,40 detik dan pengiriman notifikasi darurat di bawah satu detik. Dengan demikian, FIRESENTRY memenuhi aspek dependability, real-time processing, dan reliability sebagai sistem embedded yang andal untuk mendukung keamanan lingkungan.

5.2 SARAN

Untuk pengembangan lebih lanjut, disarankan agar sistem dilengkapi dengan mekanisme penyimpanan data lokal menggunakan SD card atau flash storage sehingga log data tetap tersimpan meskipun koneksi internet terputus. Selain itu, penambahan fitur pemutus aliran listrik otomatis berbasis relay merupakan prioritas pengembangan yang paling krusial, di mana relay akan dikendalikan langsung oleh ESP32 ketika sistem mendeteksi kondisi bahaya sehingga risiko korsleting listrik sebagai pemicu kebakaran dapat segera dieleminasi secara otomatis tanpa memerlukan intervensi manual. Kalibrasi sensor MQ-2 menggunakan nilai PPM yang terstandarisasi juga perlu dilakukan agar pembacaan konsentrasi gas menjadi lebih presisi. Dari sisi perangkat lunak, pengembangan antarmuka dashboard berbasis web atau aplikasi mobile yang terhubung langsung ke MQTT broker akan membuat proses monitoring lebih nyaman dan informatif bagi pengguna. Terakhir, pengujian sistem pada lingkungan nyata dengan variasi kondisi yang lebih beragam perlu dilakukan untuk memvalidasi keandalan sistem secara menyeluruh sebelum diterapkan pada skala yang lebih besar.